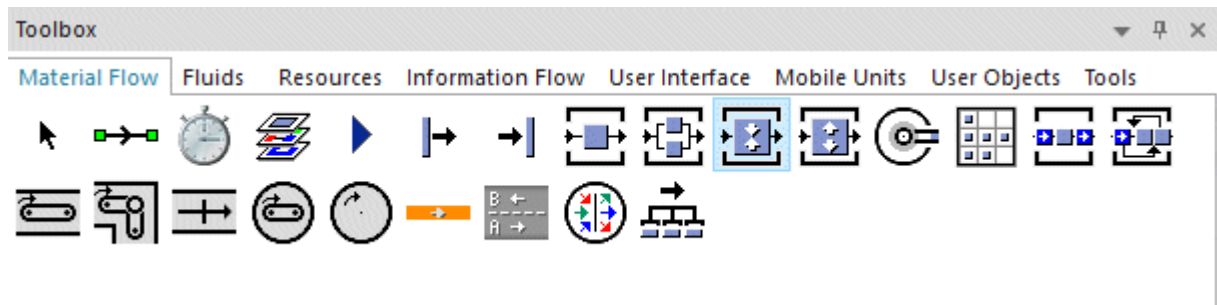


Assembling Parts with the Assembly Station

Assemble Parts with the AssemblyStation

The *AssemblyStation* adds mounting parts to a main part, for example doors to a car body, or legs to a table top as in our example. It moves the mounting parts either to the main *MU*—according to the value you enter into the *Assembly Table*—or it deletes them.

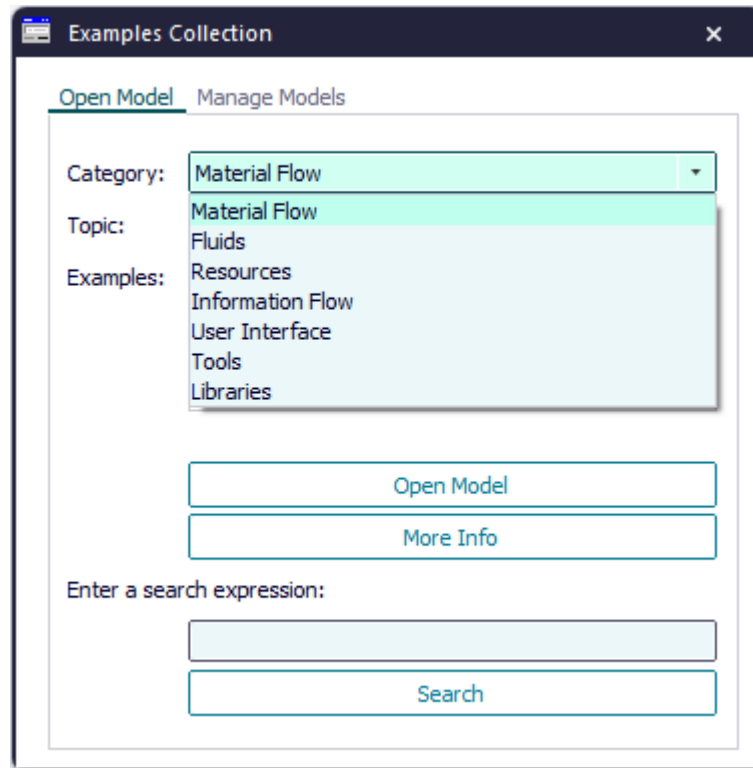
You can insert the **AssemblyStation** into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.



In our examples we will:

- **Assemble Kitchen Tables**
- **Always Fill-up the Main MU with Parts**
- **Assemble Parts Sequentially**
- **Assemble Kitchen Tables**

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Back to [Modeling the Material Flow](#)

Assemble Kitchen Tables

Assemble Kitchen Tables

In this basic sample model we demonstrate how to produce two different types of tables and show how to attach the table legs to the table tops.

The *Source* named *TableTopsIn* produces two brown table tops and two white table tops. The *Source* named *TableLegsIn* produces four table legs with different colors. The sources send the table tops and table legs on to the *AssemblyStation* which in turn attaches the respective legs to the matching table tops. It then sends the assembled tables on to shipping station, which we modeled with a *Drain*.

We will demonstrate how to:

- [Create the Assembly of the Kitchen Tables](#)
- [Adjust the Graphics of the Assembly](#)

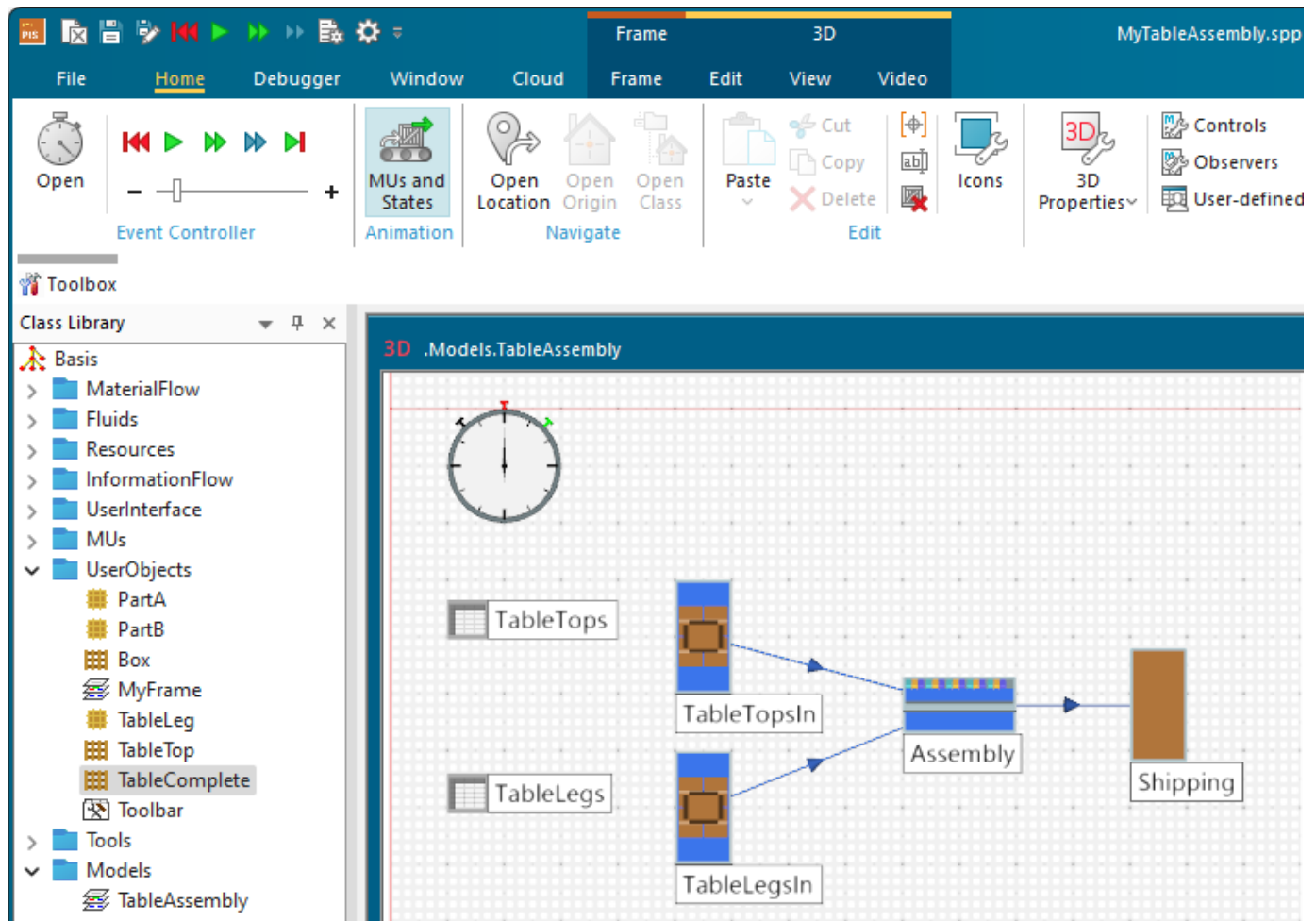
Compare [Change the Assembly Table in the Entrance Control](#)

Back to [Assemble Parts with the AssemblyStation](#)

Create the Assembly of the Kitchen Tables

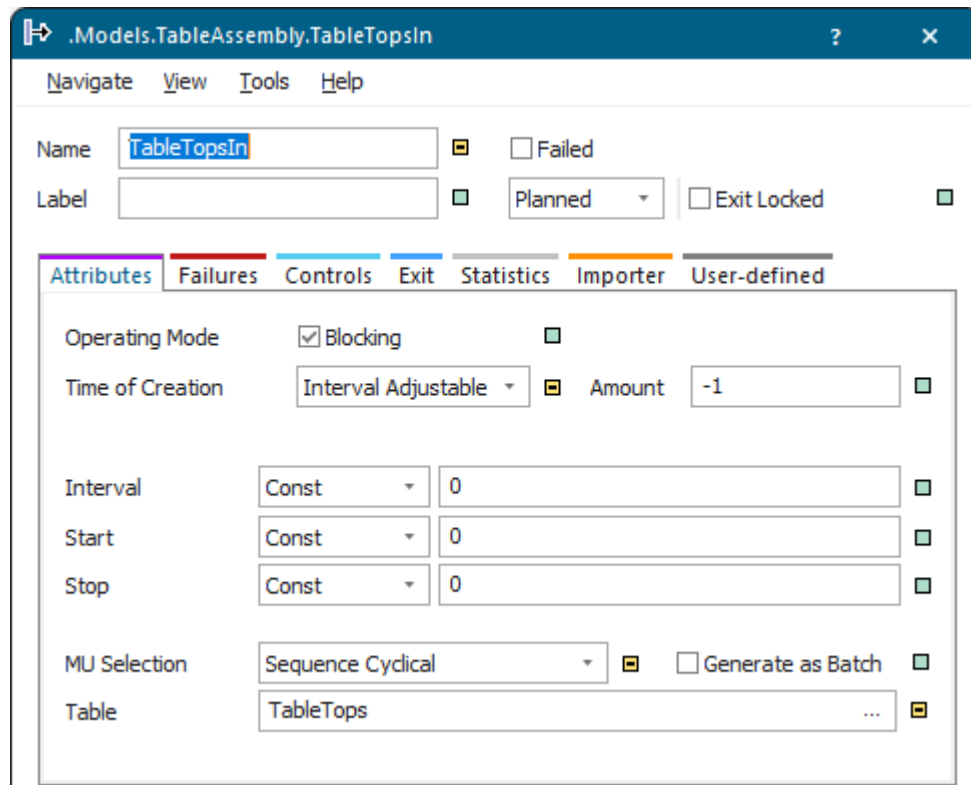
We inserted and configured the *Sources*, which produce the parts, the table tops and table legs. The *AssemblyStation* then assembles the table.

To create the simulation model, we inserted the objects so that they match the screenshot below.



We inserted and configured the *Sources*, which are to produce the parts, the table tops and table legs in our case:

- We selected the type of part which the *Source* named *TableTopsIn* is to produce. We selected the **MU Selection > Sequence Cyclical** and then selected the name of the **Sequence Table**, which we inserted before, namely the *DataTable* named *TableTops*. Selecting the table automatically applied the correct format.



- We entered the required data into the **Sequence Table**.

We dragged the **MU** named `TableTop`, which we created and stored in the folder **Models**, to the cell **MU** and dropped it there.

As we want to create two brown table tops, we entered 2 as the **Number**. We then entered the name of the part, namely `TableTop_A`, which also is the name of the icon of the brown table top into the **Attributes** subtable, which we named `a1`.

	object 1	integer 2	string 3	table 4
string MU		Number	Name	Attributes
1	.Models.TableTop	2	TableTop_A	a1
2	.Models.TableTop	2	TableTop_B	a2
3				
4				
5				
6				

	string 1	integer 2	boolean 3	string 4
string Name of Attribute				
1	CurrIcon			TableTop_A
2	PictureName3DTableTop			TableTop_A

We repeated this for our white table top.

- We selected the type of part which the source *TableLegsIn* is to produce. We selected the **MU selection** > **Sequence Cyclical** and then selected the name of the **Sequence Table**, which we inserted before, namely the *DataTable* named *TableLegs*. Selecting the table automatically applied the correct format.

The screenshot shows the configuration window for the assembly *.Models.TableAssembly.TableLegsIn*. The **Attributes** tab is selected. The **Name** field is set to *TableLegsIn*. The **Label** field is empty. The **Operating Mode** is set to **Blocking** (checked). The **Time of Creation** is set to **Interval Adjustable** with an **Amount** of **-1**. The **Interval**, **Start**, and **Stop** values are all set to **Const** with a value of **0**. The **MU Selection** is set to **Sequence Cyclical**. The **Table** field is set to **TableLegs**. There are checkboxes for **Failed**, **Planned**, **Exit Locked**, and **Generate as Batch**.

- We entered the required data into the **Sequence Table**.

We dragged the **MU** named *TableLeg*, which we created and stored in the folder **Models**, to the cell **MU** and dropped it there.

As we want to create two sets of legs for our brown table tops, we entered 8 as the **Number**. We then entered the name of the part, namely *Leg_A* into the **Attributes** subtable, which we named *a1*.

The background window, titled `.Models.TableAssembly.TableLegs`, displays a table with the following data:

	object 1	integer 2	string 3	table 4
string	MU	Number	Name	Attributes
1	.Models.TableLeg	8	Leg_A	a1
2	.Models.TableLeg	8	Leg_B	a2
3				
4				
5				
6				
7				
8				
9				

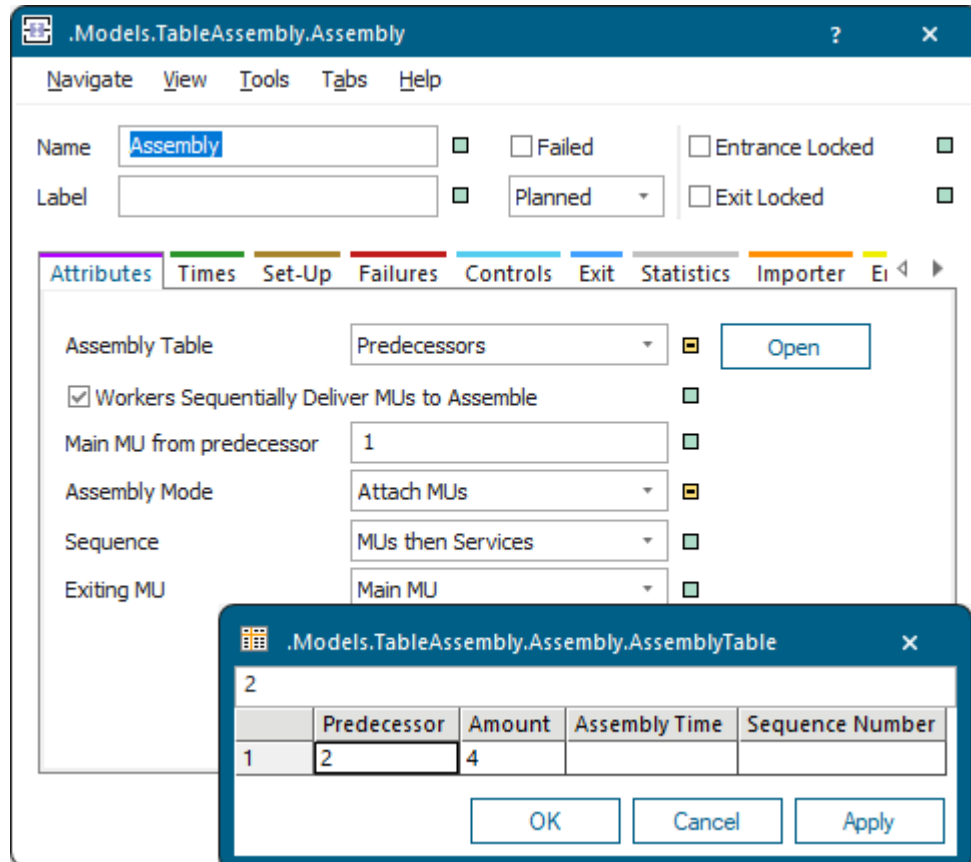
The foreground window, titled `.Models.TableAssembly.TableLegs[4, 1]`, displays a table with the following data:

	string 1	integer 2	boolean 3	string 4	real 5
string	Name of Attribute				
1	Curlicon			Leg_A	
2	PictureName3D			Leg_A	

We repeated this for the legs that go with our white table top.

We inserted and configured the *AssemblyStation*, which is going to attach the matching table legs to the table tops.

- As we want to attach the table legs to the table tops, we selected **Assembly Table > Predecessors**.
- We then clicked **Open** and entered the number of the **predecessor** from which the legs arrive, 2 in our case, and the number of parts want to attach, 4 in our case.



- As the **Assembly Mode** we selected **Attach MUs**.
- As the part that the *AssemblyStation* is going to pass on is the finished table, we selected **Main MU**, which arrived **from predecessor 1**.

When we start the simulation, we can watch how the *AssemblyStation* attaches the four blue legs to the brown table top, passes the assembled table on to the shipping station, and repeats this once. It then attaches four orange legs to the white table top, passes the assembled table on to the shipping station, and does the same again, and then repeats the sequence, namely two brown table tops, followed by two white table tops, and so on.

Note

If the assembly process does not work as expected, check the sequence in which you connected its predecessors with the *AssemblyStation*. Dragging the mouse over a *Connector* shows the number of the *Connector* in the tooltip. Also make sure that the **Main MU** arrives **from the correct predecessor**.

Back to [Assemble Kitchen Tables](#)

Adjust the Graphics of the Assembly

We have to adapt the dimensions of the table legs and of the table top to make them look somehow realistic and to adapt it to the standard graphic of our *AssemblyStation*.

- For the table legs we entered these settings for the **MU Size** for the class of the **TableLeg**.

.Models.TableLeg

Name: ☐ Stopped

Label: Conveying Direction:

Attributes | Routing | Product Statistics | Costs | User-defined

MU Size

Length	0.14	☐ m
Width	0.14	☐ m
Height	0.74	☐ m

Booking Point

0.5	☐ [0..1]	(0.07 m)
0.5	☐ [0..1]	(0.07 m)
0	☐ [0..1]	(0.00 m)

- For the table tops we entered these settings for the **MU Size** for the class of the **TableTop**.

.Models.TableTop

Name: ☐ Stopped

Label: Conveying Direction:

Attributes | Routing | Product Statistics | Statistics | Costs | User-defined

MU Size

Length	1	☐ m
Width	1	☐ m
Height	0.1	☐ m

Booking Point

0.5	☐ [0..1]	(0.50 m)
0.5	☐ [0..1]	(0.50 m)
0	☐ [0..1]	(0.00 m)

X-Dimension: ☐

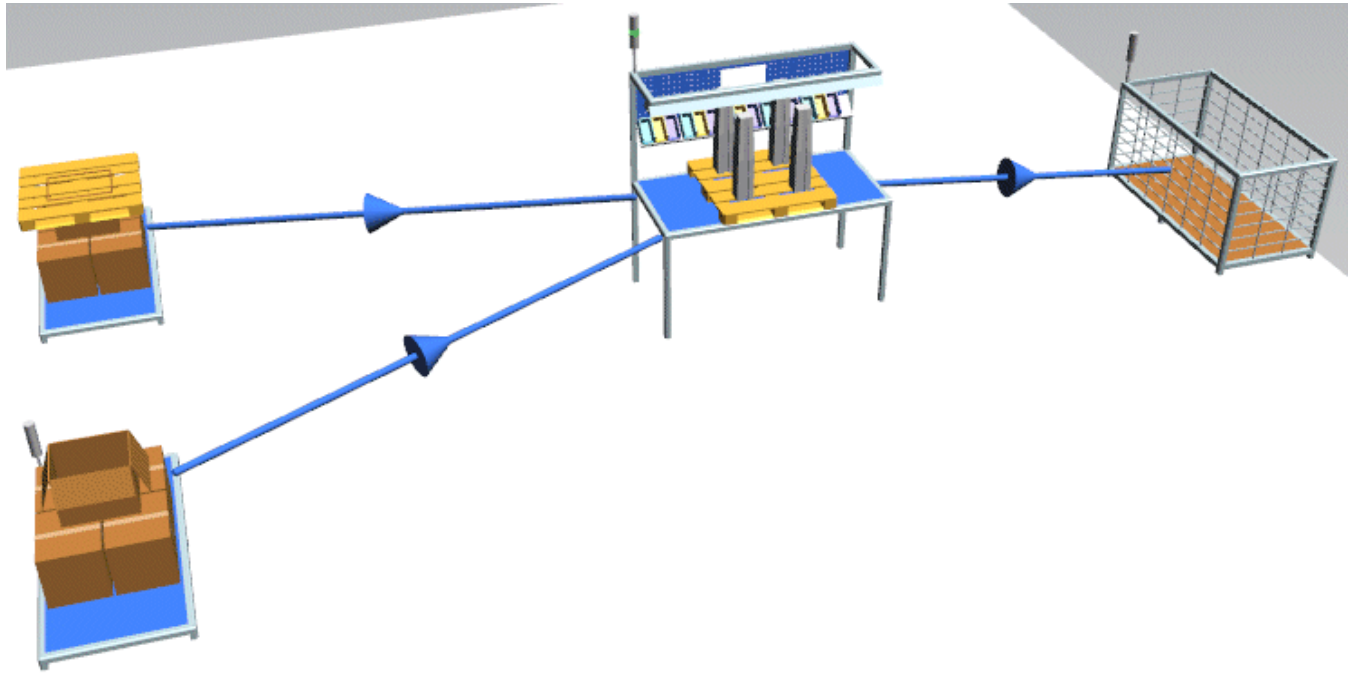
Y-Dimension: ☐

Z-Dimension: ☐

☐ Workpiece Carrier ☐

☐ Fill Whole Layer ☐

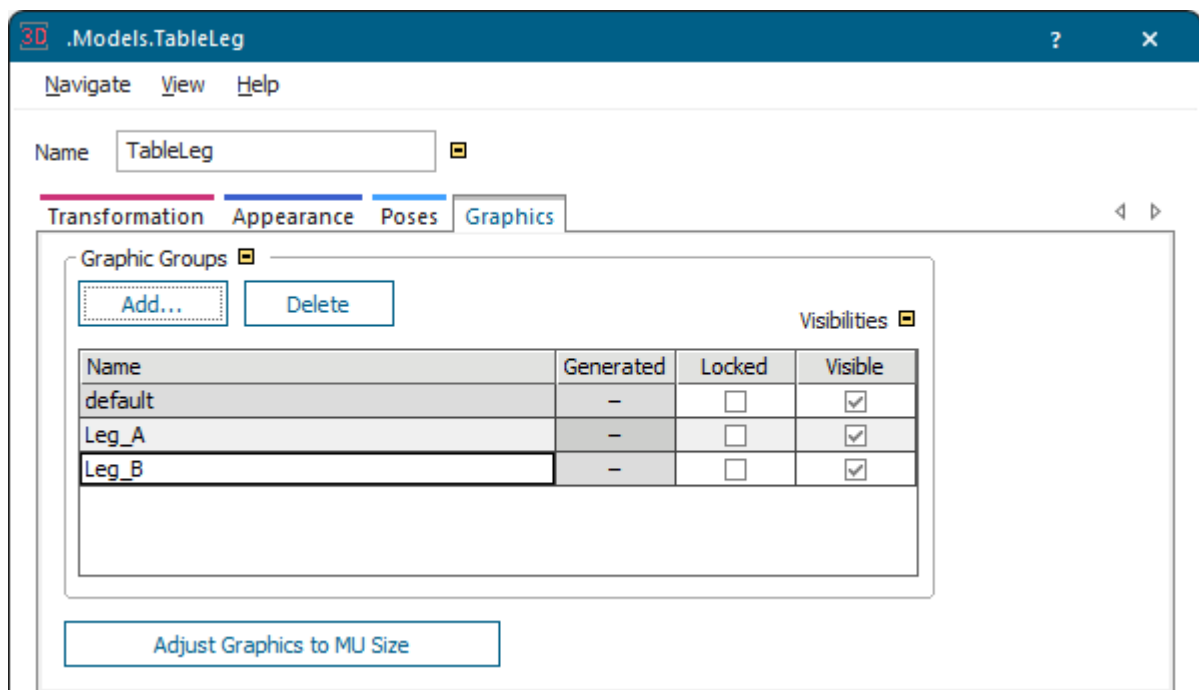
- The result with the adjusted dimensions of the standard graphics of the part and the pallet look like this:



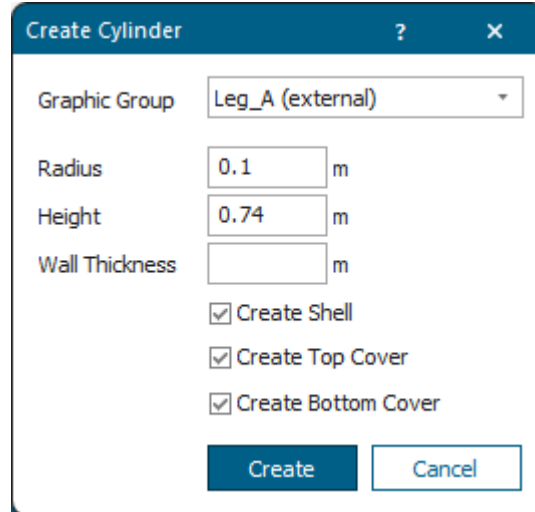
As our tables do not look like tables, we create graphics for the varicolored table legs and table tops.

Proceed as follows to create the blue and orange table legs:

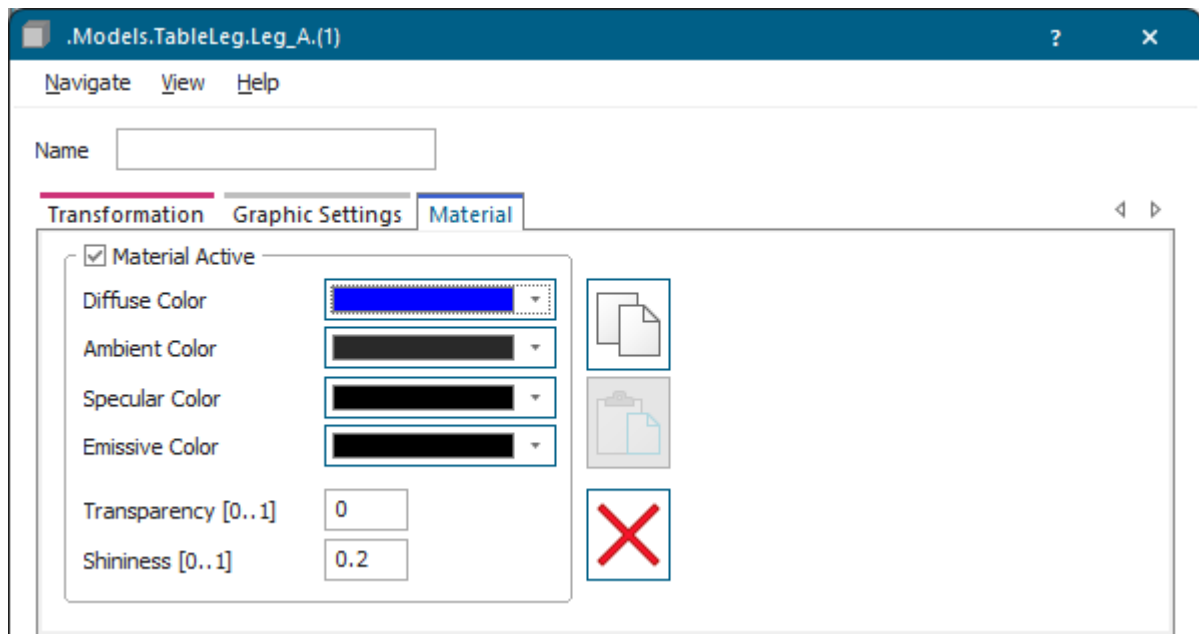
- Right-click the object **TableLeg** in the *Class Library* and select **Open in 3D**.
- Press the spacebar and change to the tab **Graphics** in the dialog **Edit 3D Properties**. Add a graphic group with the name **Leg_A** and **Leg_B** each.



- Delete the graphic named **default**. Change to the **Edit** tab and click **Cylinder** under **Insert Shape**. Select the graphic group for which you want to create the graphic. We selected these settings for **Leg_A**:



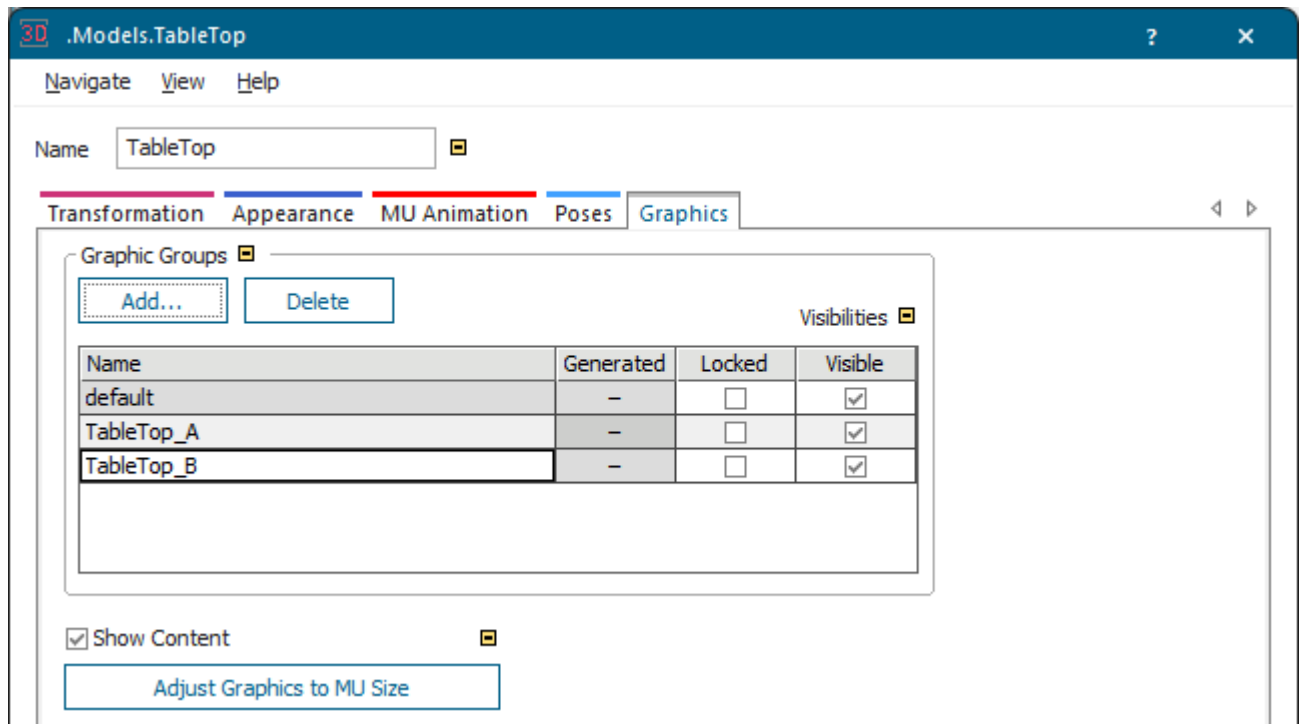
- This inserts the *Cylinder* at the origin of the scene. It does not have the right color though.
- Click the *Cylinder* and press the spacebar. Change to the **Material** tab, select **Material Active**, and select a color. Our **Leg_A** is blue.



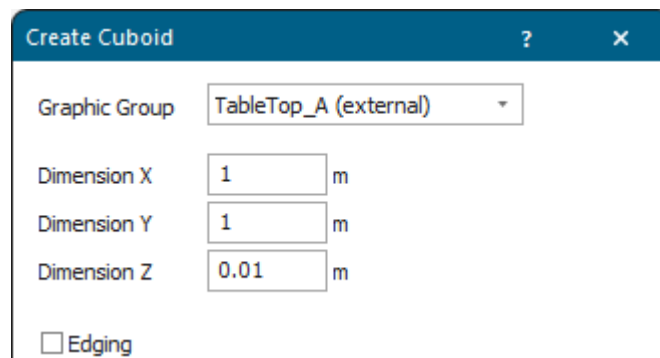
- Repeat these steps for **Leg_B** and select **Orange** as the color.

Proceed as follows to create the brown and the white table tops:

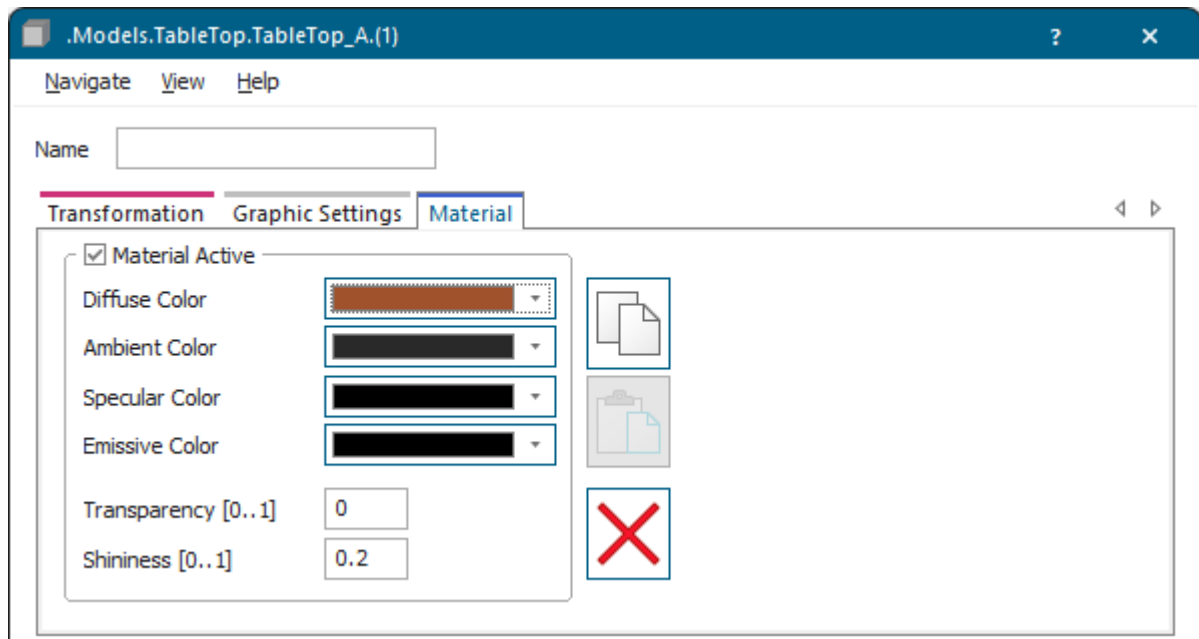
- Right-click the object **TableTop** in the *Class Library* and select **Open in 3D**.
- Press the spacebar and change to the tab **Graphics** in the dialog **Edit 3D Properties**. Add a graphic group with the name **TableTop_A** and **TableTop_B** each.



- Delete the graphic named **default**. Change to the **Edit** tab and click **Cuboid** under **Insert Shape**. Select the graphic group for which you want to create the graphic. We selected these settings for **TableTop_A**:



- This inserts the *flat cuboid* at the origin of the scene. It does not have the right color yet.
- Click the *Cuboid* and press the spacebar. Change to the **Material** tab, select **Material Active**, and select a color. Our **TableTop_A** is brown.

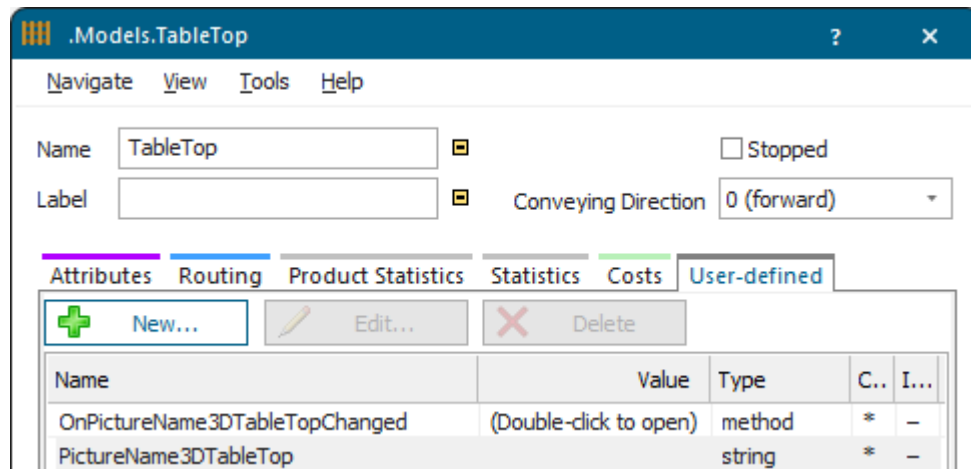


- Repeat these steps for **TableTop_B** and select a non all white tone.

When we run the simulation, we see that the *AssemblyStation* assembles the varicolored tables correctly but does not show them in the correct colors.

Plant Simulation shows them according to the graphic group names that we entered. To accomplish this, we create a *user-defined attribute* and an **observer** for the *user-defined attribute* in the class of the object.

- In the class of the table leg we created an observer for the user-defined attribute named **PictureName3D**, which detects when a table leg with another name arrives, and then displays the correct graphic.



- We entered this source code into the method of the observer **OnPictureName3DChanged**:

```
// param attribute: string, oldValue: string, newValue: string
// param attribut: string, oldValue: string
```

```
param newValue: string
self.~._3D.VisibleGraphicGroups := [newValue]
```

- We then enter the name of the user-defined attribute named *PictureName3D* and the name of the respective graphic group into the two sub-tables named **a1** and **a2** of the *DataTable* named *TableLegs*.

object 1	integer 2	string 3	table 4
string MU	Number	Name	Attributes
1 .Models.TableLeg	8	Leg_A	a1
2 .Models.TableLeg	8	Leg_B	a2
3			
4			
5			
6			
7			
8			

string 1	integer 2	boolean 3	string 4	real 5
string Name of Attribute				
1 PictureName3D			Leg_A	
2				

- In the class of the table top we created an observer for the user-defined attribute *PictureName3DTableTop*, which detects when a table top with another name arrives, and then displays the correct graphic.

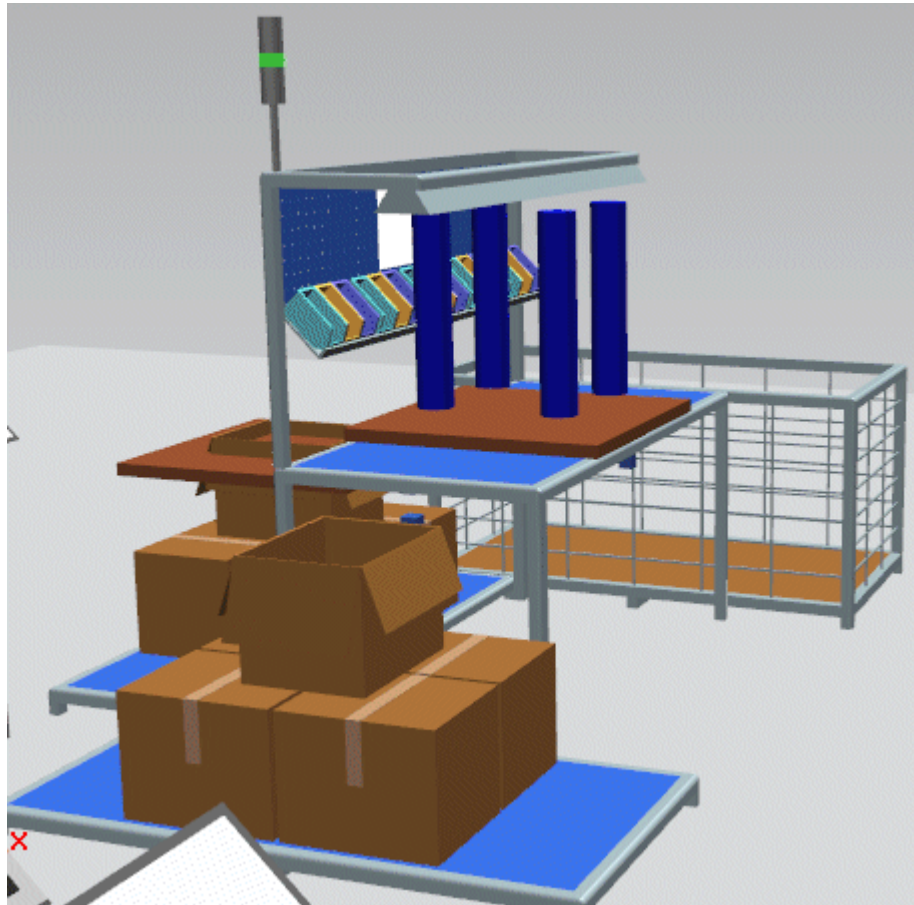
Name	Value	Type	C..	I...
OnPictureName3DTableTopChanged	(Double-click to open)	method	*	-
PictureName3DTableTop		string	*	-

- We entered this source code into the method of the observer **OnPictureName3DTableTopChanged**:

```
// param Attribute: string, oldValue: string, newValue: string
// param Attribut: string, oldValue: string
param newValue: string
self.~._3D.VisibleGraphicGroups := [newValue]
```

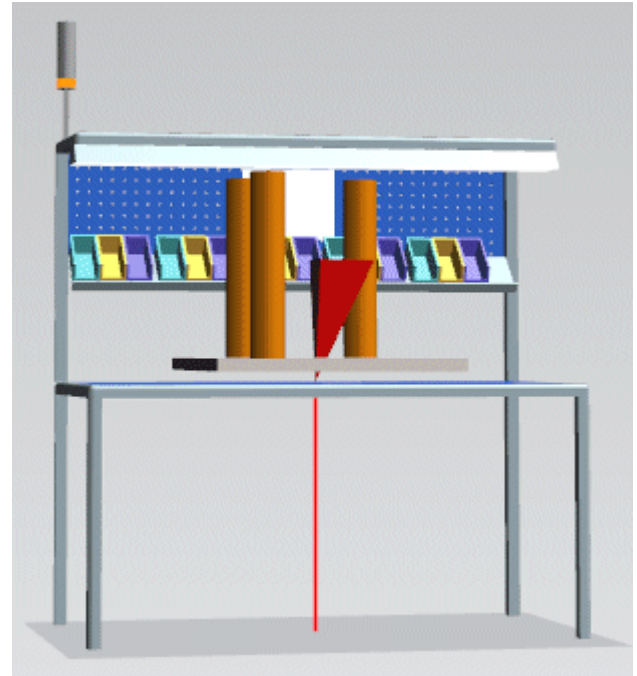
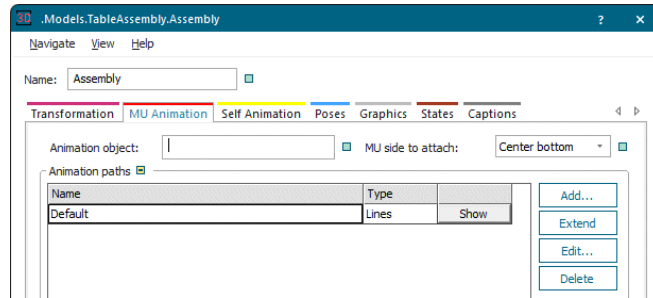
- We then enter the name of the user-defined attribute named *PictureName3DTableTop* and the name of the respective graphic group into the two sub-tables named **a1** and **a2** of the *DataTable* named *TableTops*.

When we now run the simulation, we see that two tables each with a brown and two tables each with a bright-colored table top are produced. When zooming the *AssemblyStation* we see that the table top is not positioned flat on the assembly area, but with an offset to the top. In addition the left legs jut into the assembly boxes at the back of the *AssemblyStation*.

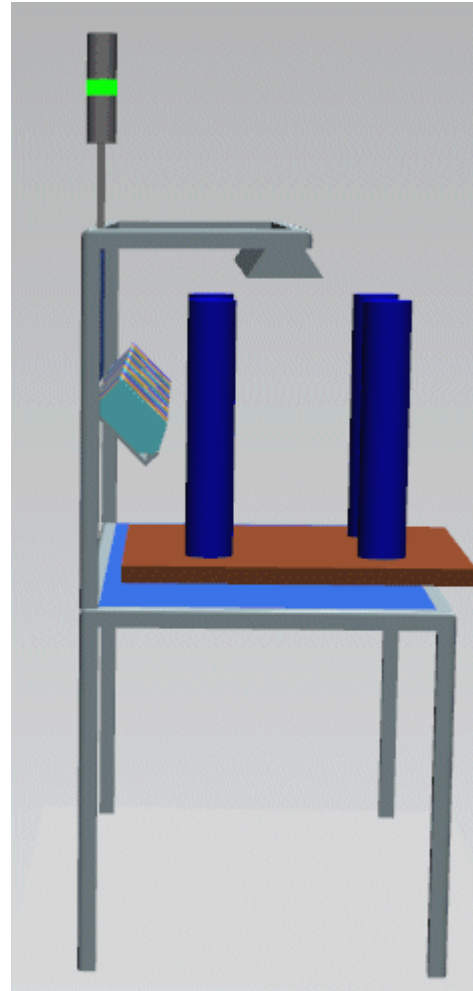
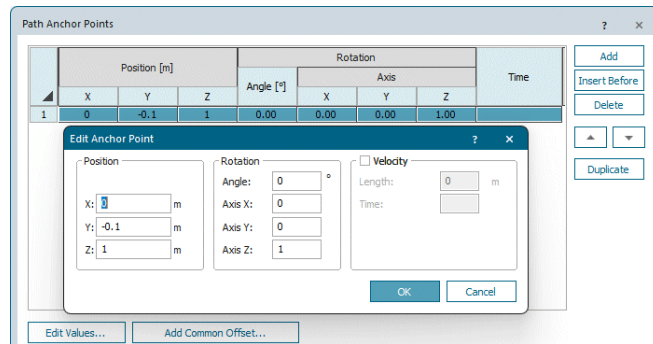


This is easy to correct:

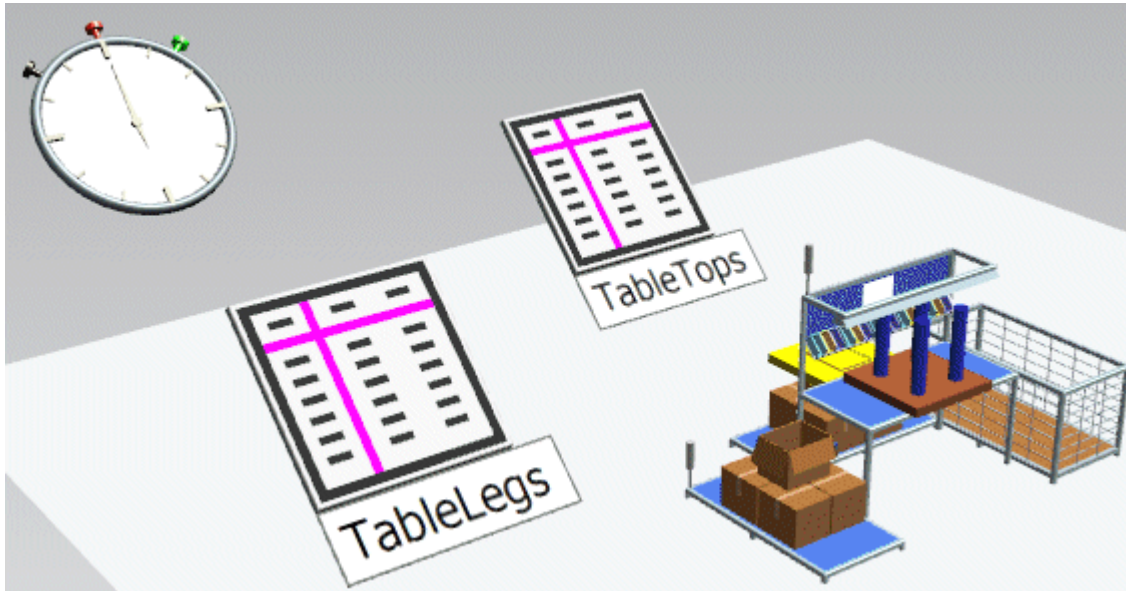
- Click the *AssemblyStation* and press the spacebar. Change to the tab **MU Animation**.
- Show the path named **default**.



- Click **Edit** and change the values as shown in the figure so that the table top lies flat on the assembly table and that the legs do not touch the assembly boxes any longer.



Finally we arrange the objects more realistically and run the simulation. The finished simulation model looks like this in 3D:



Back to [Assemble Kitchen Tables](#)

Always Fill-up Main MU with Parts

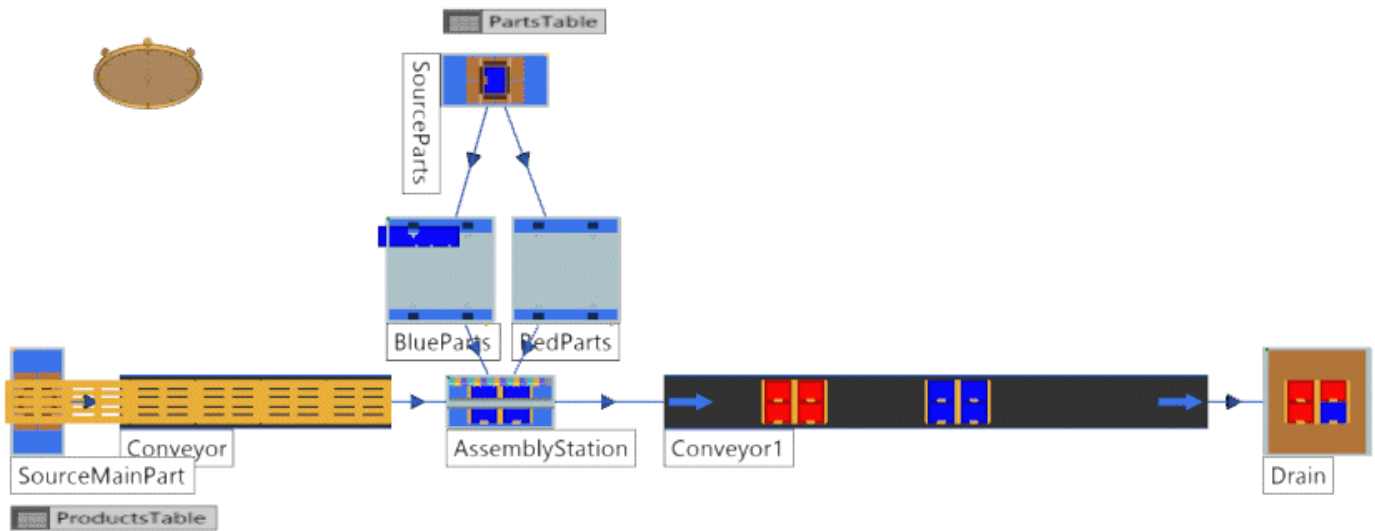
Always Fill-up the Main MU with Parts

In this basic sample model we demonstrate how to always fill up the **main MU**, here a pallet, with as many parts as it can hold. The part type that is loaded onto the pallet does not matter.

We will demonstrate how to:

- [Configure the Material Flow to Fill-up Main MU](#)
- [Configure the Assembly Table to Fill-up the Main MU](#)

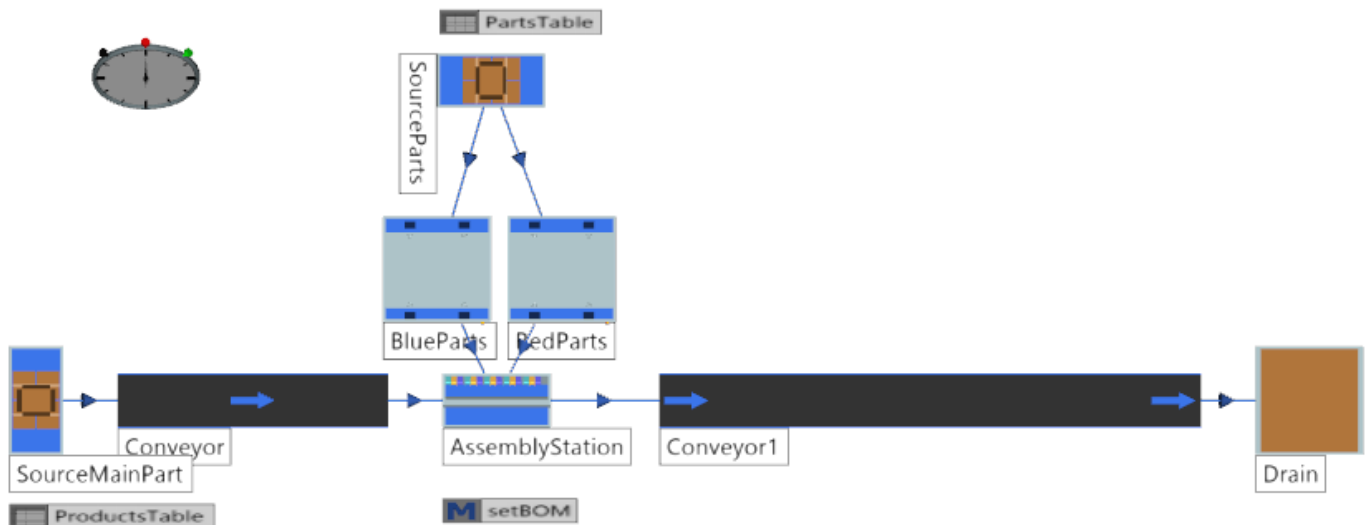
Our finished model looks like this:



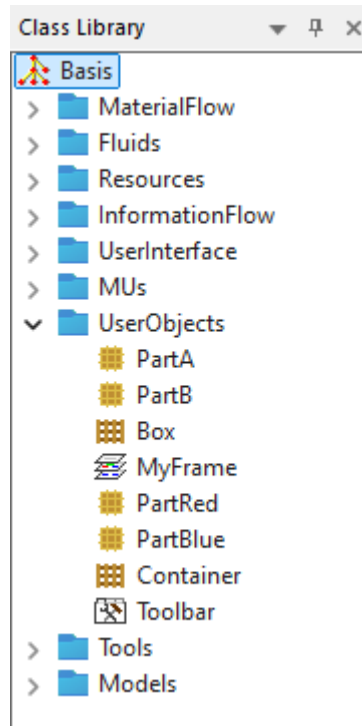
Back to Assemble Parts with the AssemblyStation

Configure the Material Flow to Fill-up Main MU

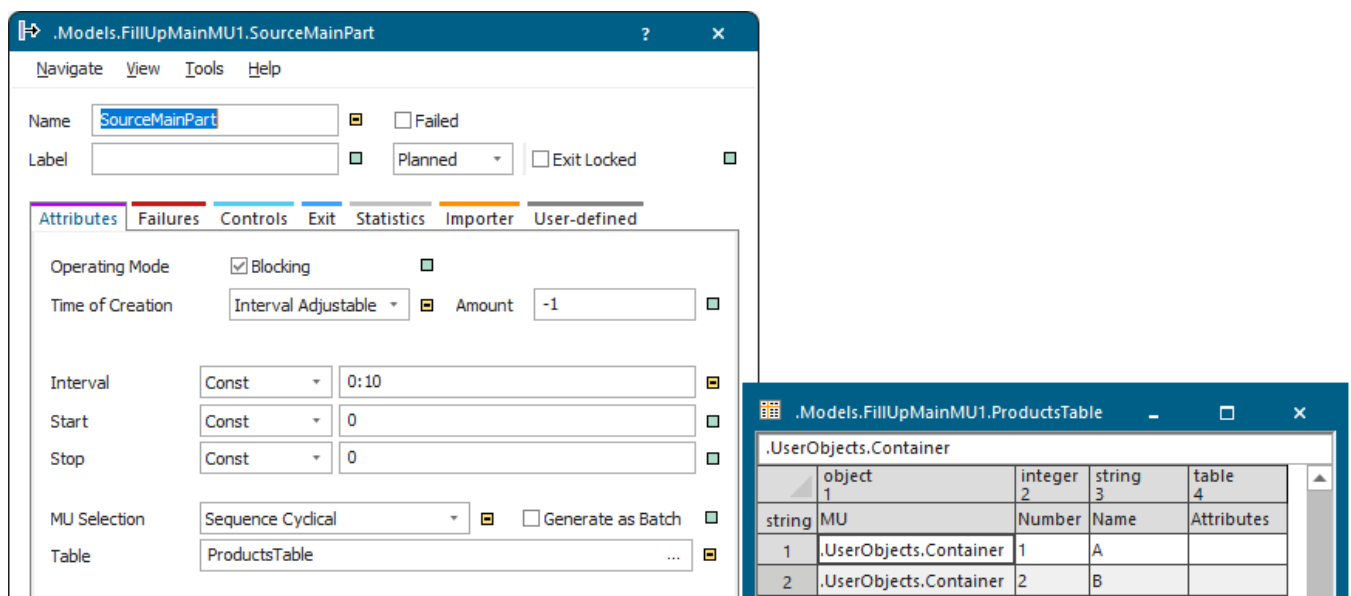
Insert the material flow objects, so that they match the screenshot below. Make sure that all objects are connected. Then configure the material flow objects.



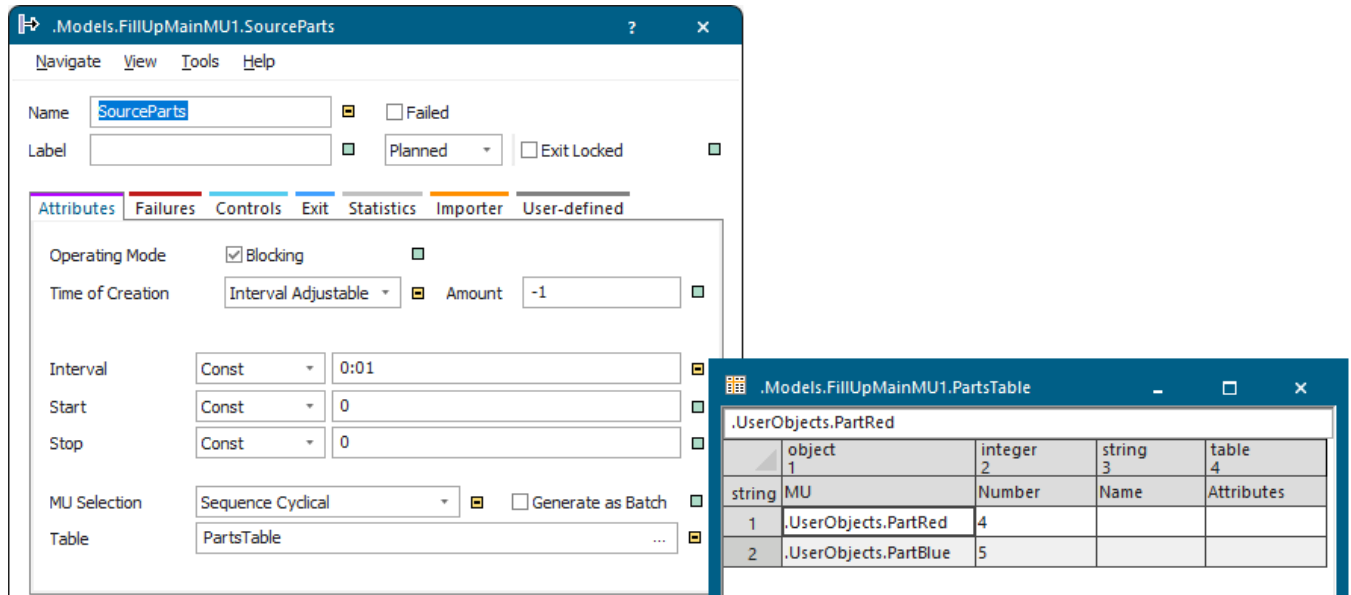
- Our first step in configuring the material flow to fill-up the **main mu** is to duplicate the *Part* twice. We named the duplicated parts **PartRed** and **PartBlue** and assigned them the respective colors. We also duplicated the *Container*.



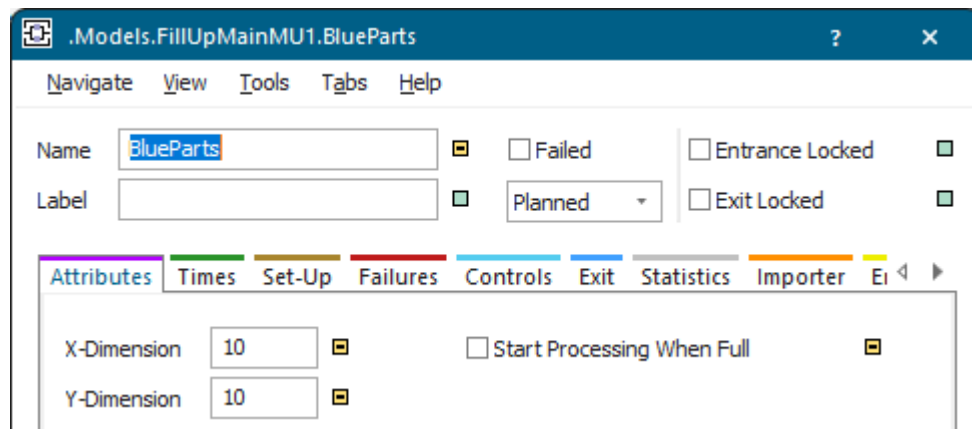
- The Source named *SourceMainParts* produces two types of *Containers*, as specified in the *ProductsTable*, in a sequential sequence.



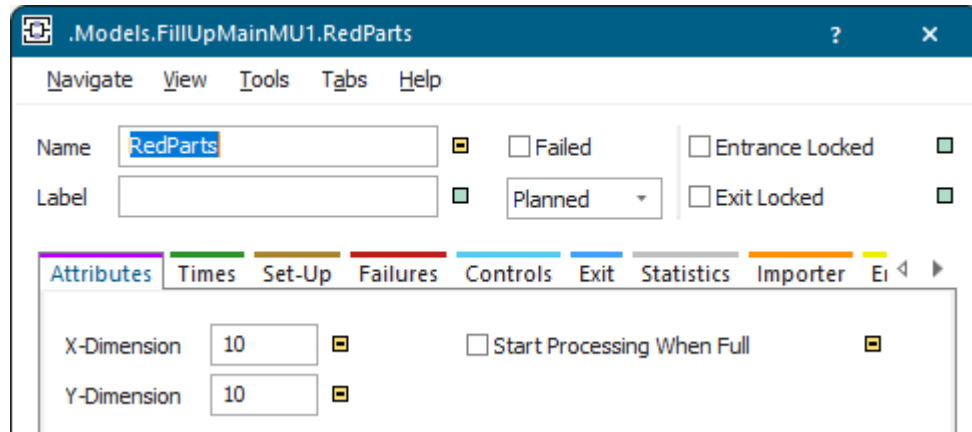
- The *SourceMainParts* sends the pallets along a *Conveyor* to the *AssemblyStation*.
- The Source named *SourceParts* produces two types of *Parts* with different colors, as specified in the *PartsTable*, in a cyclical sequence.



- The *SourceParts* sends the *PartBlue* on to the *ParallelStation* named *BlueParts*. We entered the settings below. We cleared **Start Processing When Full** to fill up the pallet with different part types. We also entered a processing time of 10 seconds on the **Tab Times**.



- The *SourceParts* sends the *PartRed* on to the *ParallelStation* named *RedParts*. We entered the settings below. We cleared **Start Processing When Full** to fill up the pallet with different part types. We also entered a processing time of 10 seconds on the **Tab Times**.



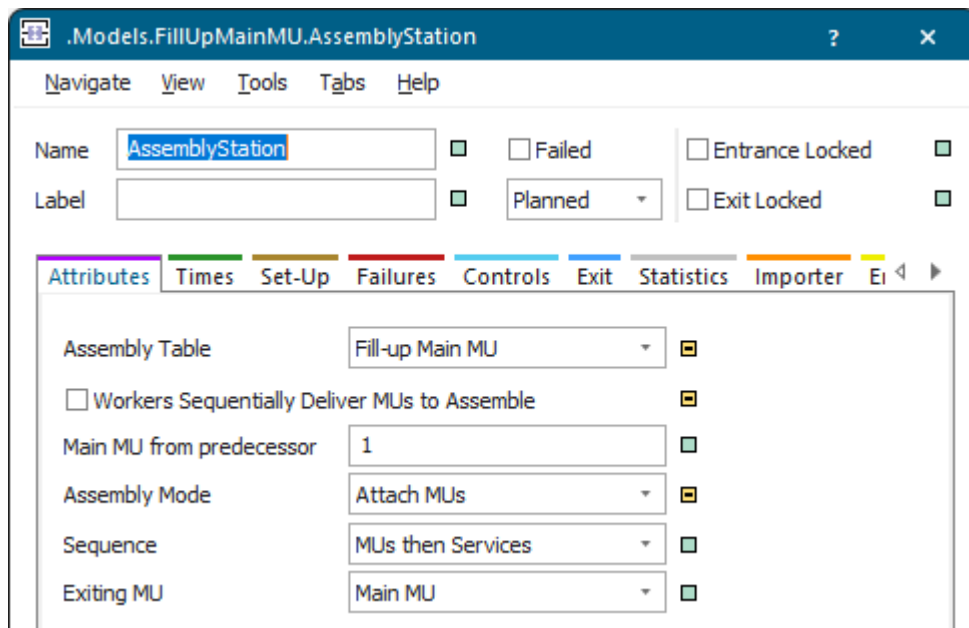
- The *ParallelStations* send the processed parts on to the *AssemblyStation* which loads them onto the pallets.

Back to Always Fill-up the Main MU with Parts

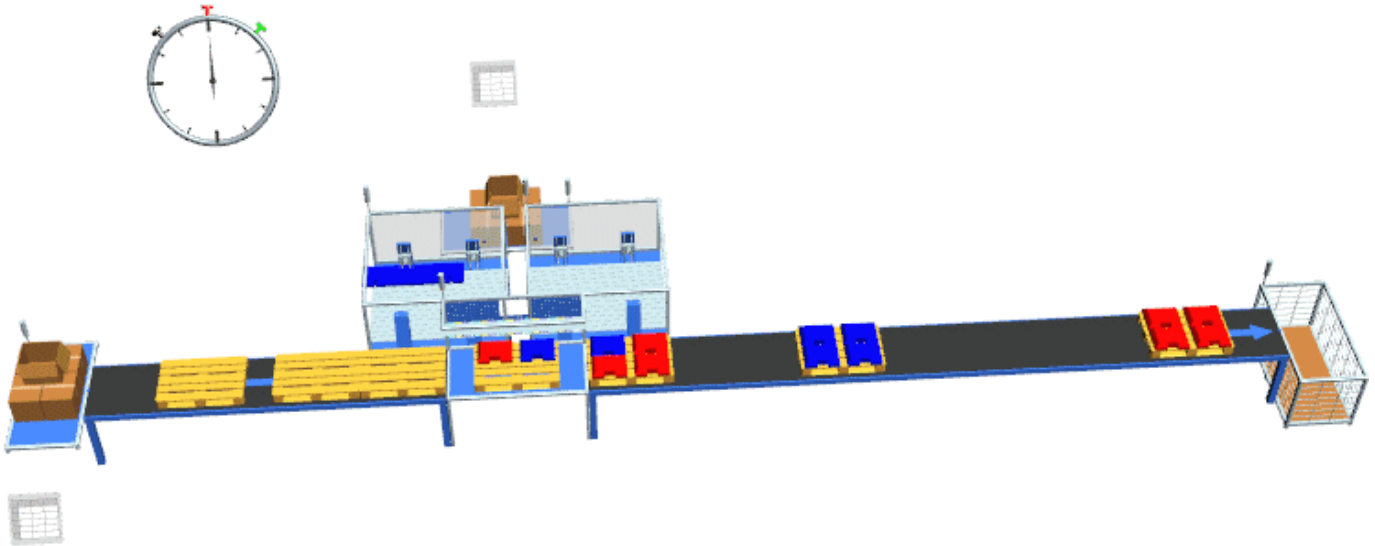
Configure the Assembly Table to Fill-up the Main MU

To make our *AssemblyStation* load the processed blue and red parts onto the pallets and always fill up the pallets to their capacity, namely four parts, we selected **Assembly Table > Fill-up Main MU**. We did not change any other settings.

Although **Fill-up Main MU** is listed as an option of the **Assembly Table**, the *AssemblyStation* does not use an **Assembly Table** at all, but accepts parts from any of its predecessors until the capacity of the **main MU**, our *Container*, is reached.



When we run the simulation, we see that the *AssemblyStation* always fills up the pallets, i.e., it places four parts on each and every pallet. Some pallets are filled up with red parts only, some with blue parts only, and some with a mix of both part types just as their processing on the *ParallelStations* is finished.

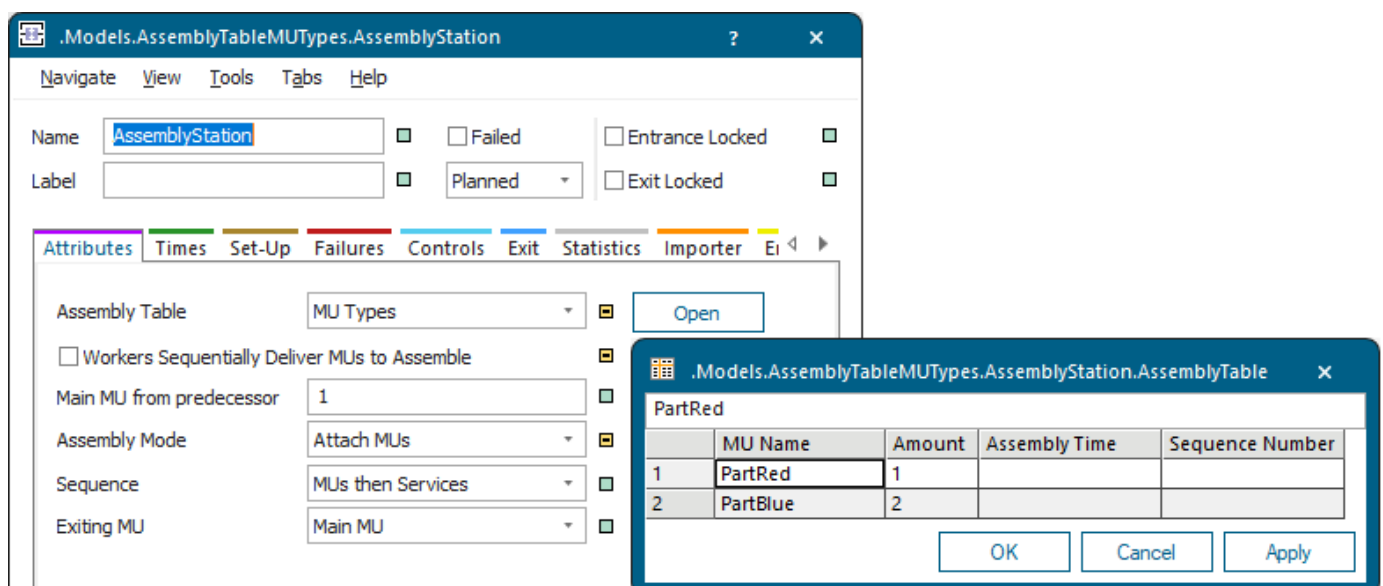


The *AssemblyStation* then sends the loaded pallets on a *Conveyor* on to the *Drain*.

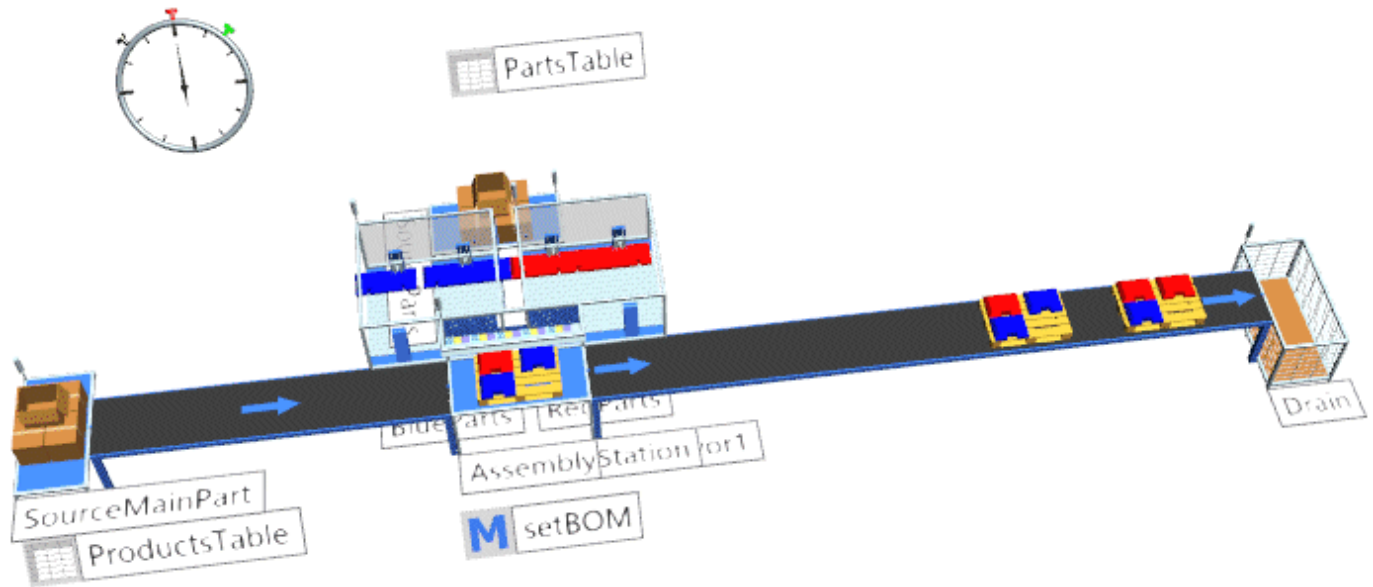
Using an Assembly Table with the Setting MU Types

Let's compare the setting **Fill-up Main MU** with the setting **Assembly Table > MU Types** in the **Example model > AssemblyStation > Change Assembly List**.

In this model the parts are moved on according to the settings specified in the **Assembly Table** and to the **Entrance Control** named *setBOM*.



When we run the simulation, we see, that the *AssemblyStation* loads the processed parts in a mixed sequence depending on the time their processing is finished onto the pallets. It never fills them up to their capacity, before sending them on to the *Drain* though.



Back to [Always Fill-up the Main MU with Parts](#)

Assemble Parts Sequentially

Assemble Parts Sequentially

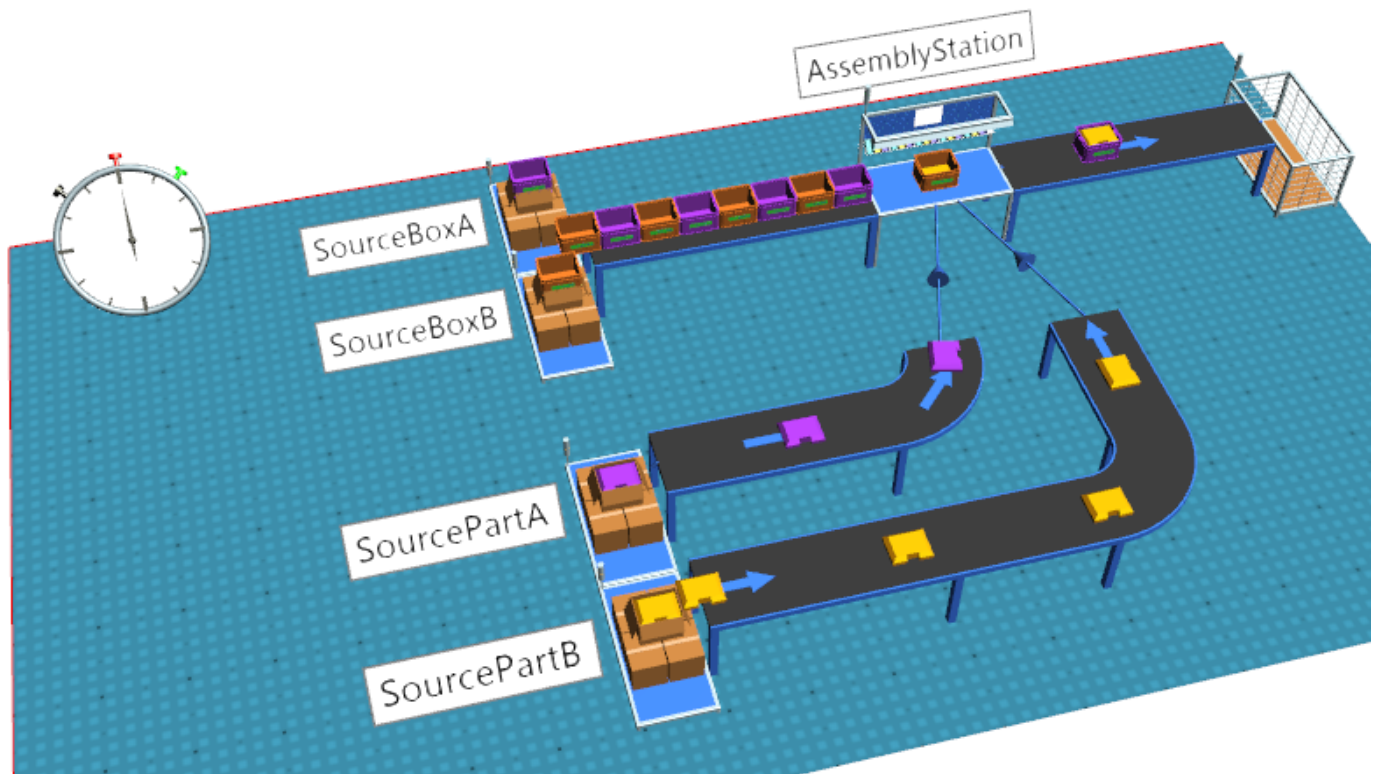
In this basic sample model we demonstrate how to **assemble parts sequentially** with the *AssemblyStation*.

To make our *AssemblyStation* assemble arriving parts sequentially, we selected **Assembly Table > Depends on Main MU**. We did not change any other settings.

We will demonstrate how to:

- [Configure the Material Flow to Assemble Parts Sequentially](#)
- [Configure the Assembly Table to Assemble Parts Sequentially](#)

Our finished model looks like this:

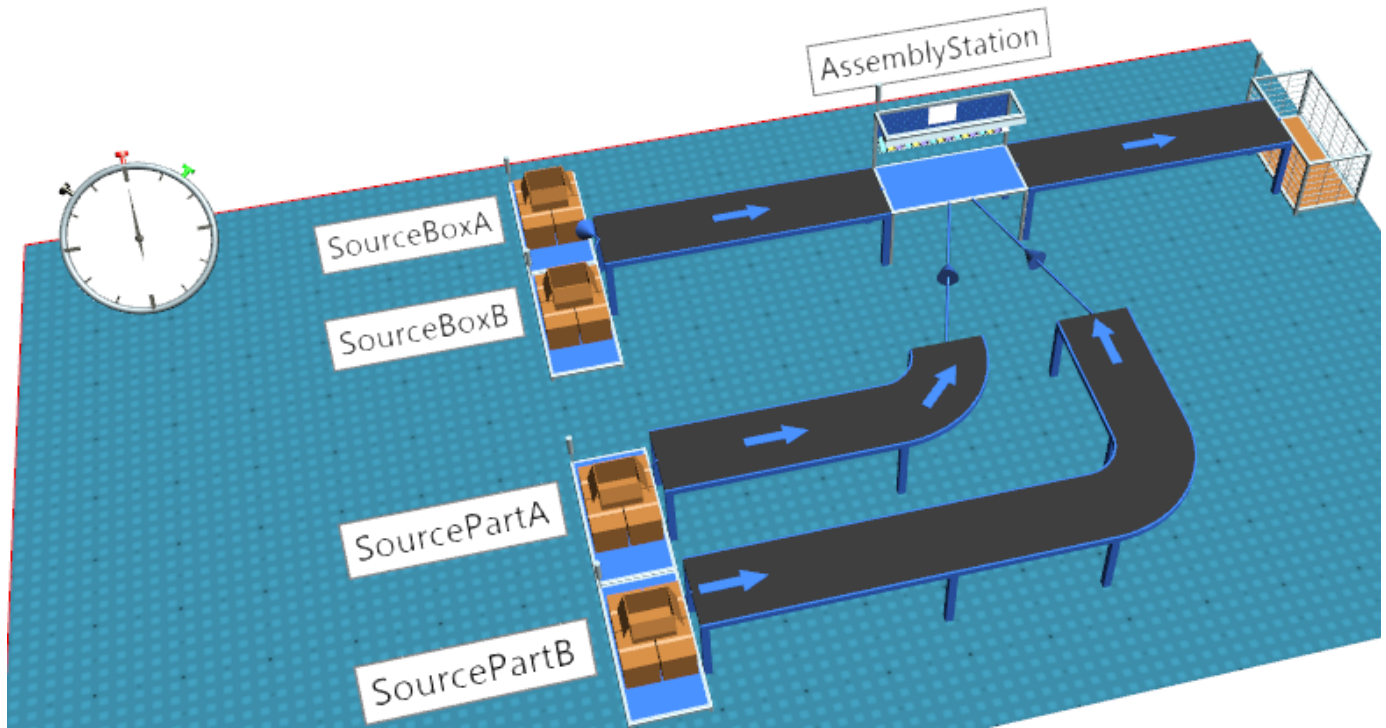


Back to [Assemble Parts with the AssemblyStation](#)

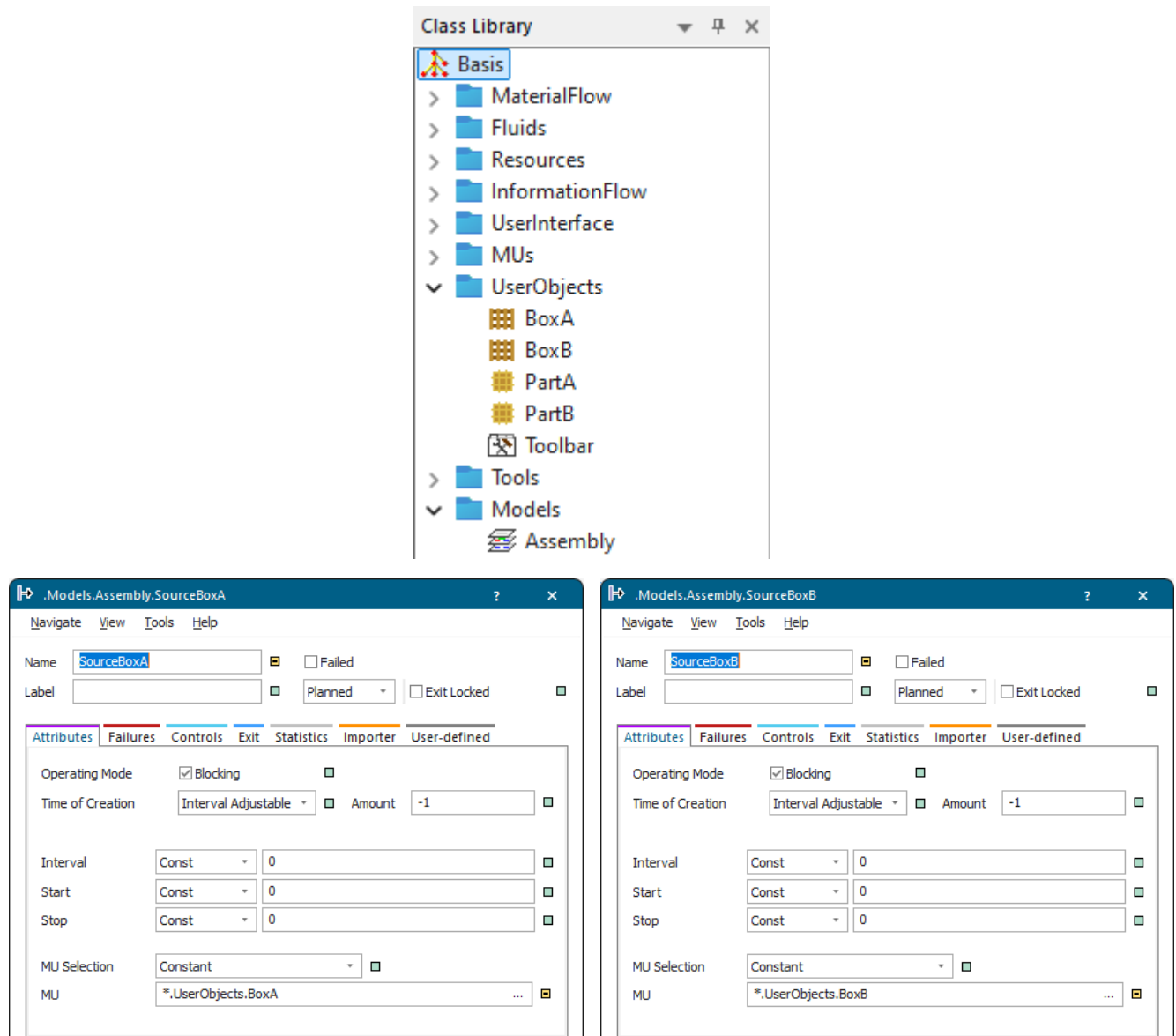
Configure the Material Flow to Assemble Parts Sequentially

Insert the material flow objects, so that they match the screenshot below. Make sure that all objects are connected and configure the material flow objects.

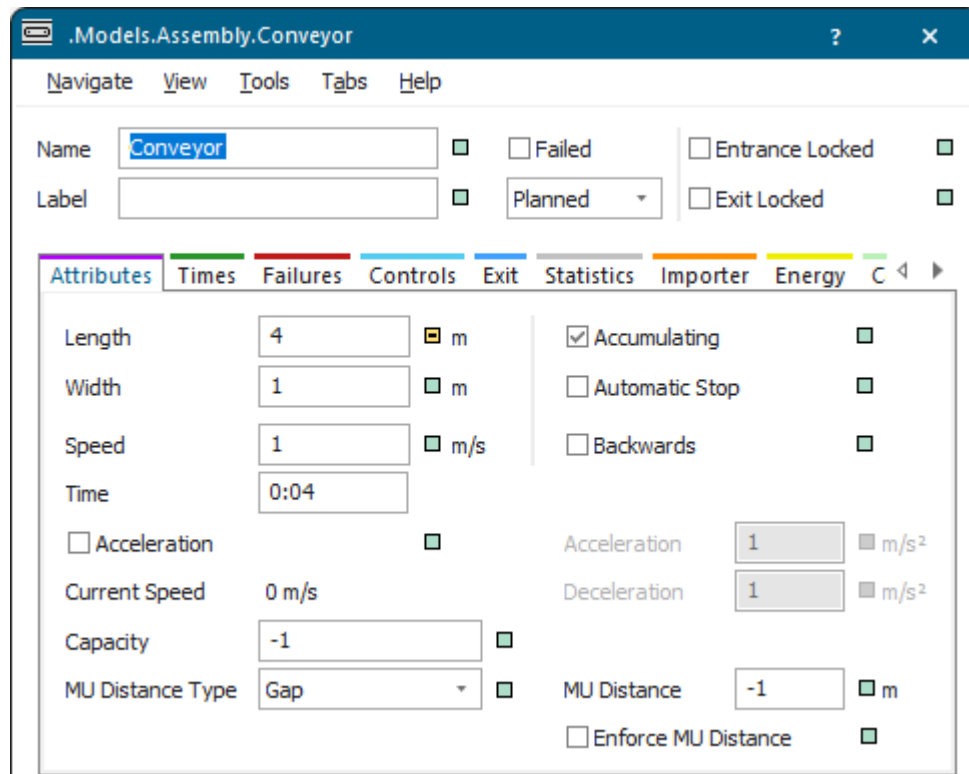
The finished simulation model looks like this:



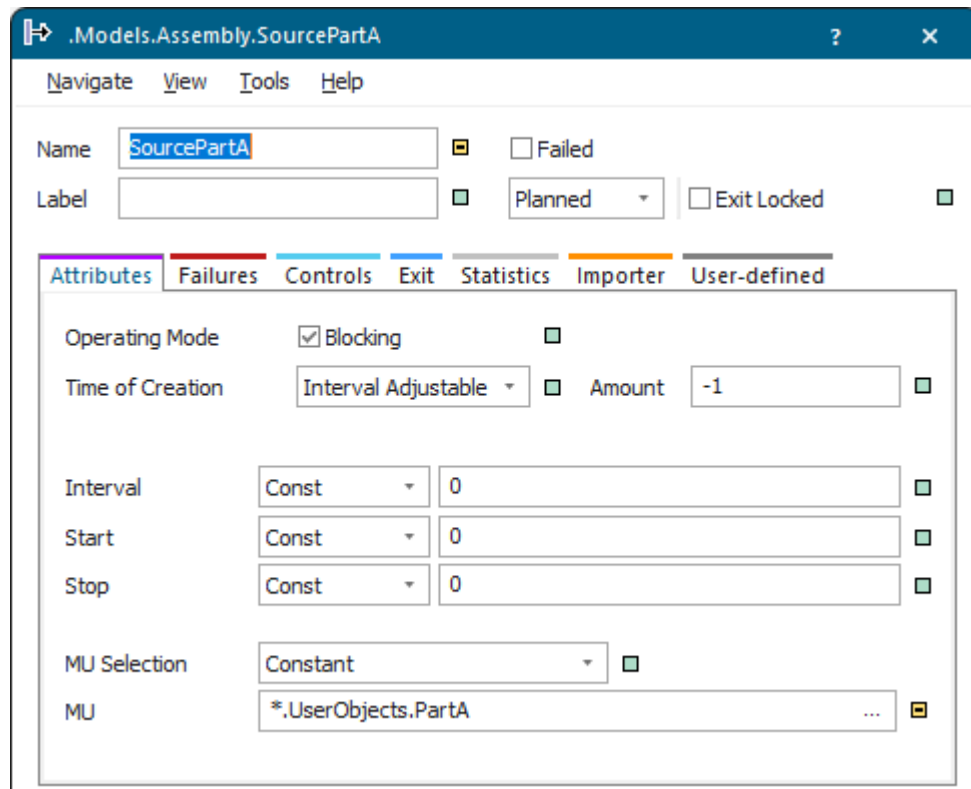
- The *SourceBoxA* produces boxes of type *BoxA* and the *SourceBoxB* produces boxes of type *BoxB*. Both are located in the folder *UserObjects* in the *Class Library*. We did not change any other settings.



- The Conveyor transporting the two types of boxes to the *AssemblyStation* is 4 meters long and uses the default settings.

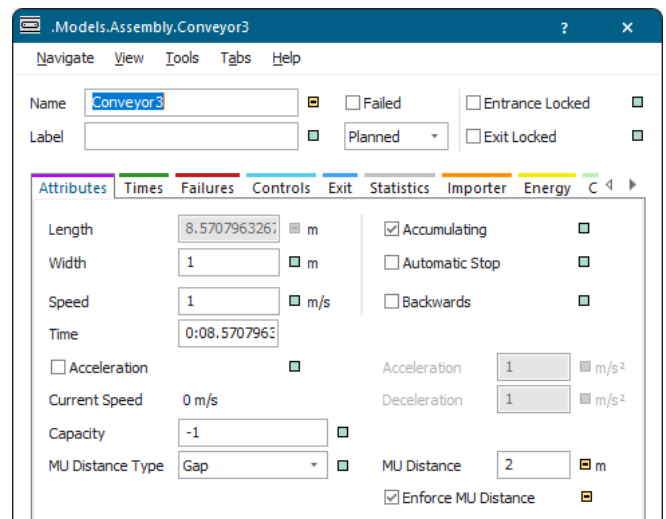
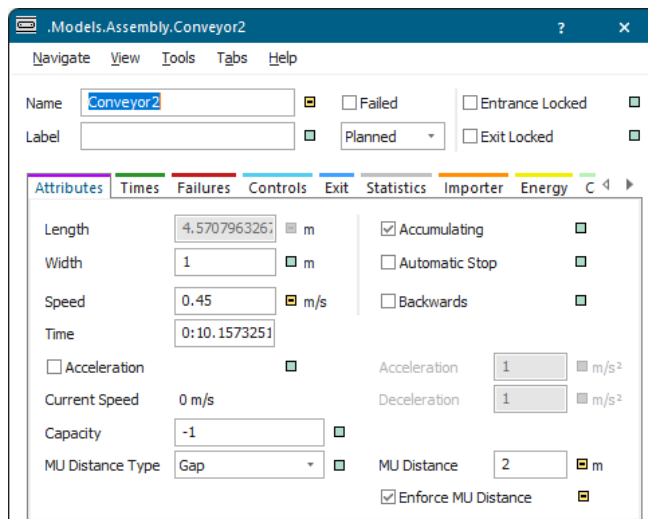


- We used the default settings for the *Conveyor* transporting the assembled parts, the two types of parts stacked in the two types of boxes, from the *AssemblyStation* to the *Drain*.
- The *SourcePartA* produces parts of type *PartA* and the *SourcePartB* produces parts of type *PartB*. We did not change any other settings.



- For the *Conveyor2* transporting parts of type *PartA* to the *AssemblyStation* we typed in a **MU Distance** of 2 meters and we selected **Enforce MU Distance**. This ensures that the *Conveyors* always keep the distance of 2 meters between the parts.

For the *Conveyor3* transporting parts of type *PartB* to the *AssemblyStation* we use the same settings.



Go to [Configure the Assembly Table to Assemble Parts Sequentially](#)

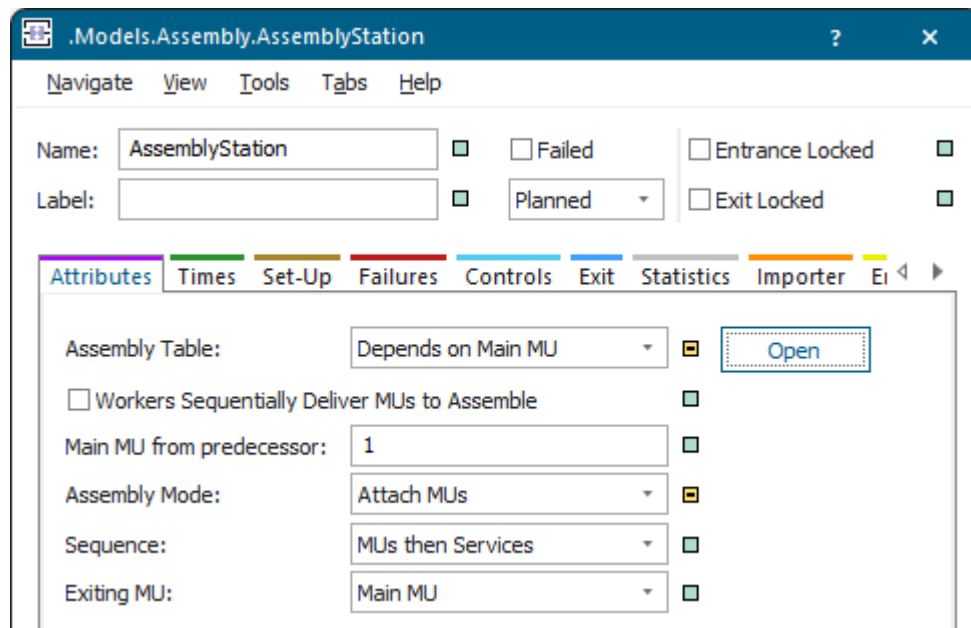
Back to [Assemble Parts Sequentially](#)

Configure the Assembly Table to Assemble Parts Sequentially

We configured the sequential assembly of the arriving parts in the **Assembly Table** of the *AssemblyStation* by typing in the **Assembly Time** and the **Sequence Number**.

We proceeded as follows:

- We selected **Depends on Main MU** and clicked **Open** to open the **Assembly Table**.



- We use these settings to assemble the parts:

BoxA					
	Main MU Name	MU Name	Amount	Assembly Time	Sequence Number
1	BoxA	PartA	1	5.0000	1
2		PartB	2	10.0000	2
3	BoxB	PartB	1	3.0000	1
4		PartA	2	6.0000	2

- We place an **Amount** of 1 *PartA* into *BoxA*. This assembly process takes an **Assembly Time** of 5 minutes . The **Sequence Number** 1 determines that *PartA* is put into the box before *PartB*.
- We place an **Amount** of 2 *PartB* into *BoxA*. This assembly process takes an **Assembly Time** of 10 minutes . The **Sequence Number** 2 determines that *PartB* is put into the box after *PartA*.
- We place an **Amount** of 1 *PartB* into *BoxB*. This assembly process takes an **Assembly Time** of 3 minutes . The **Sequence Number** 1 determines that *PartB* is put into the box before *PartA*.

- We place an **Amount** of 2 *PartA* into *BoxB*. This assembly process takes an **Assembly Time** of 6 minutes . The **Sequence Number** 2 determines that *PartA* is put into the box after *ParB*.

Run the simulation and watch how the *AssemblyStation* places the specified parts into the specified boxes in the sequence which we defined.

To follow the assembly process step-by-step, open the *Event Debugger*.

Back to [Configure the Material Flow to Assemble Parts Sequentially](#)

Back to [Assemble Parts Sequentially](#)

Assemble Parts Sequentially with the Worker

Assemble Parts Sequentially with the Worker

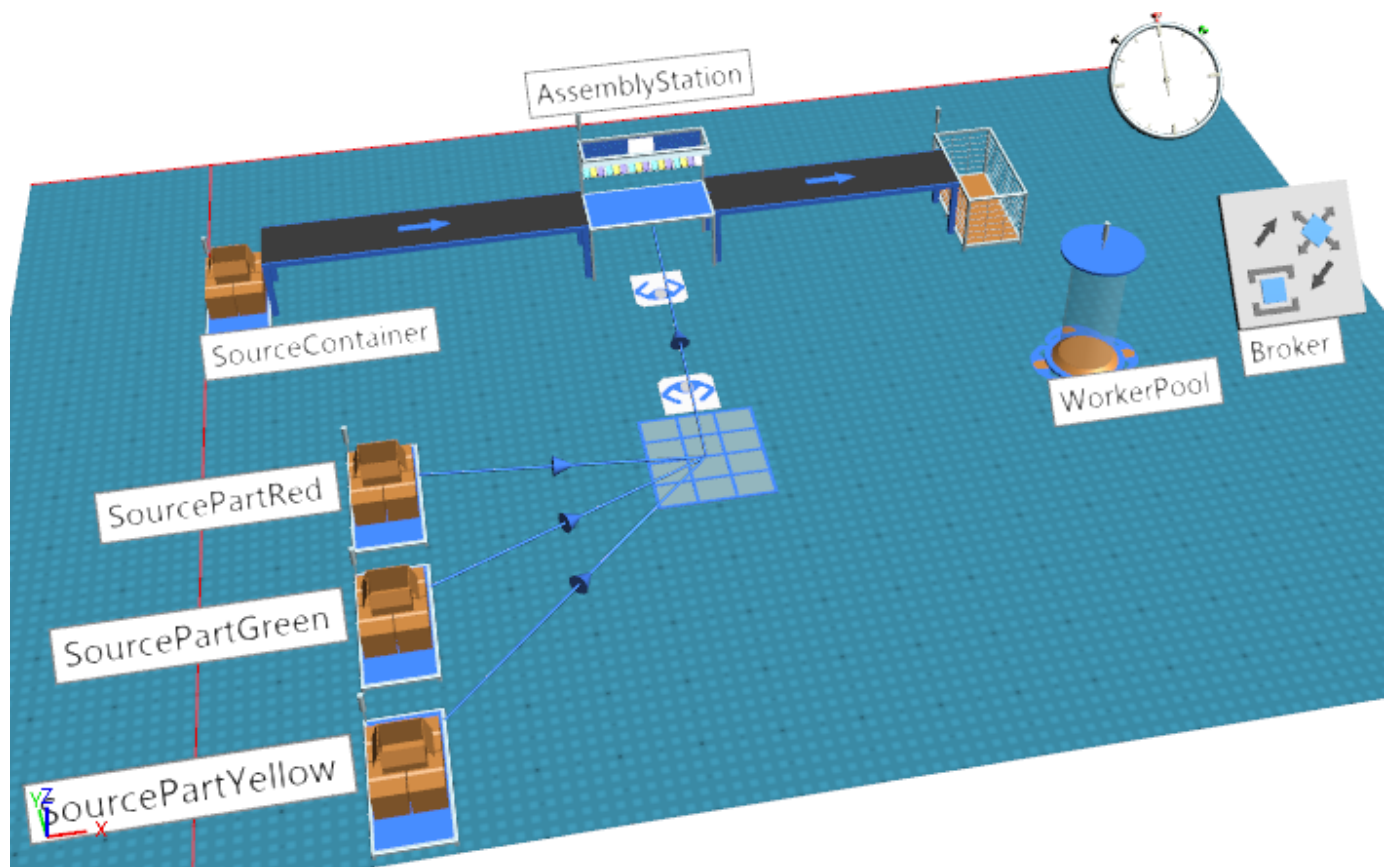
In this basic sample model we demonstrate how to make a **Worker assemble parts sequentially** with the *AssemblyStation*.

To make our *Worker* carry the parts from the *Store* to the *AssemblyStation* and place them on a pallet in the defined sequence, we selected **[Assembly Table](#) > Depends on Main MU**. We did not change any other settings.

We will demonstrate how to:

- **[Configure the AssemblyStation to Which the Worker Delivers Parts](#)**
- **[Configure the Store Which Provides the Parts](#)**
- **[Watch how the AssemblyStation Assembles the Parts Delivered by the Worker](#)**

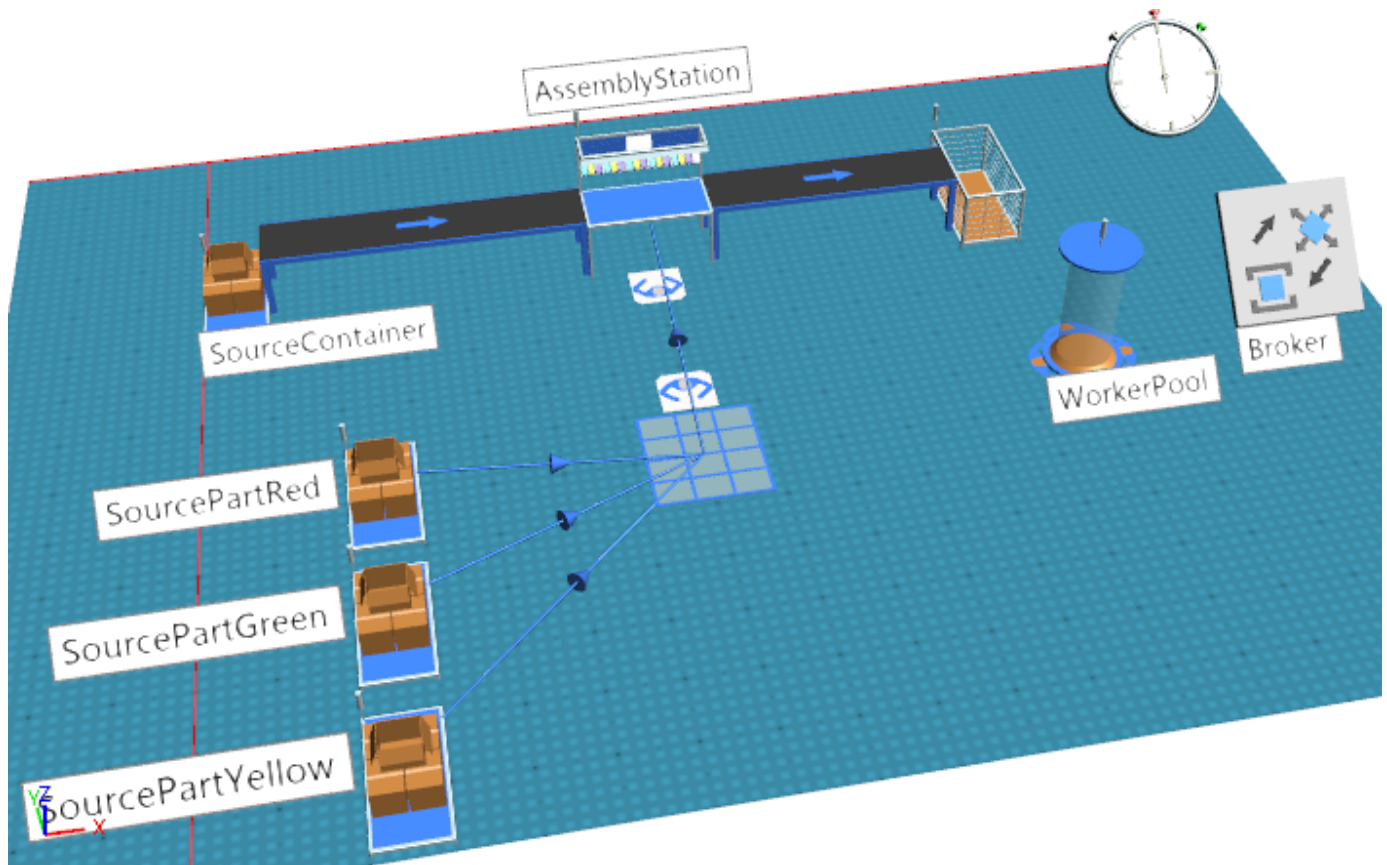
Our finished model looks like this:



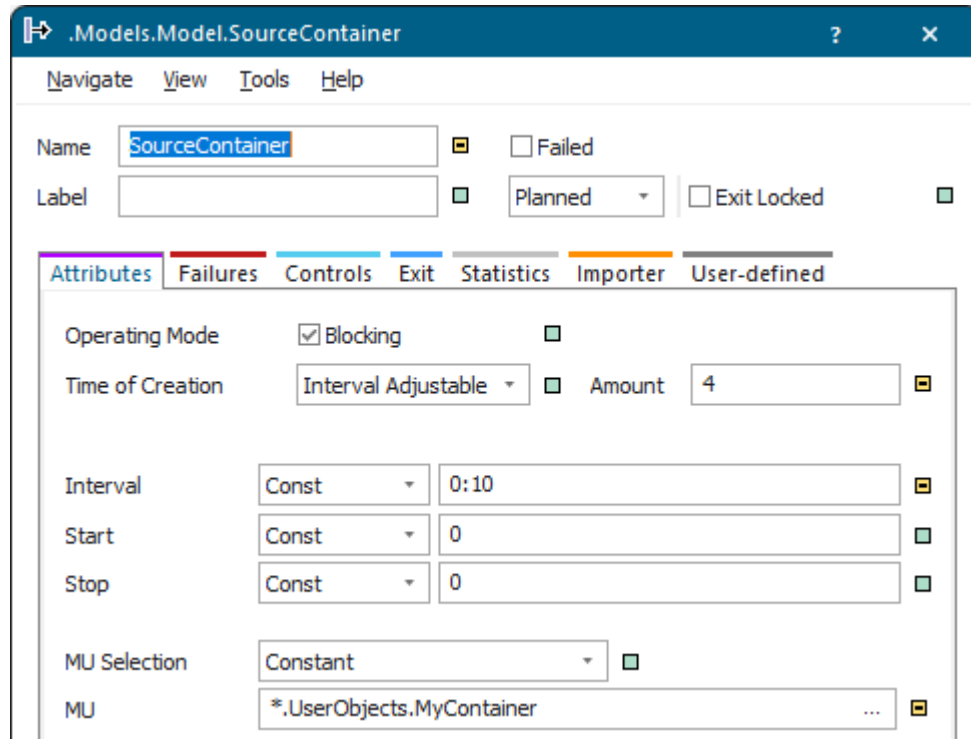
Back to [Assemble Parts with the AssemblyStation](#)

Configure the AssemblyStation to Which the Worker Delivers Parts

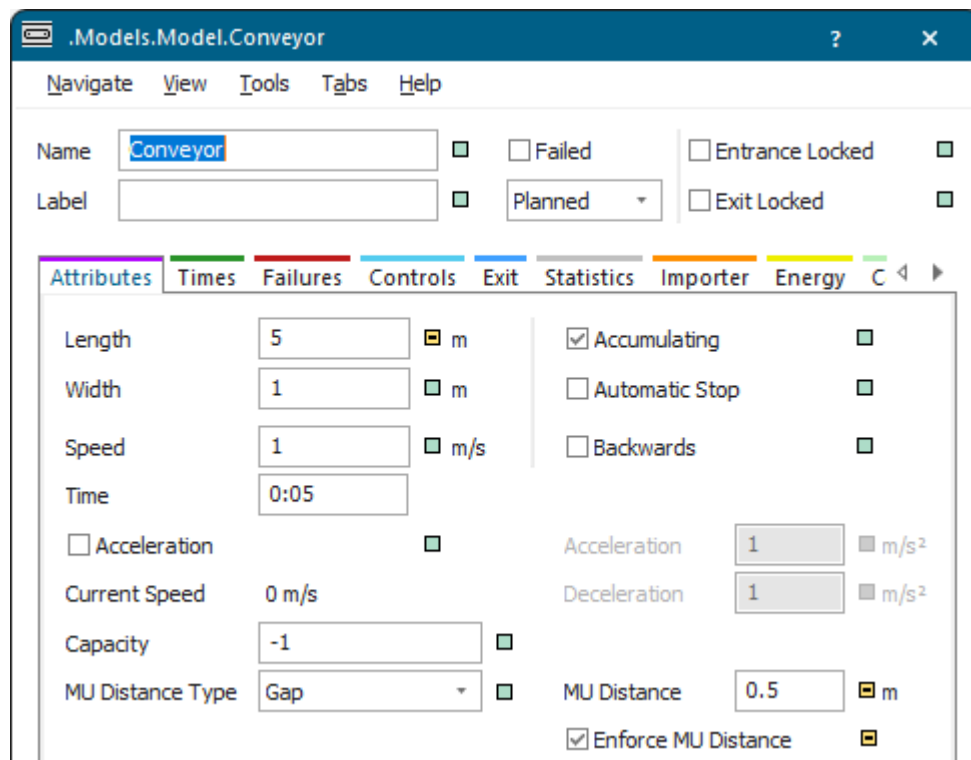
Insert the material flow objects, so that they match the screenshot below. Make sure that all objects are connected. Then configure the material flow objects.



- The *SourceContainer* produces 4 pallets onto which the *Worker* places the parts he picked up at the *Store*. We selected the settings below.



- The Conveyor transports the pallets to the AssemblyStation. As we want the pallets to keep a distance between them, we typed in 0.5 as the **MU Distance** and selected **Enforce MU Distance**.

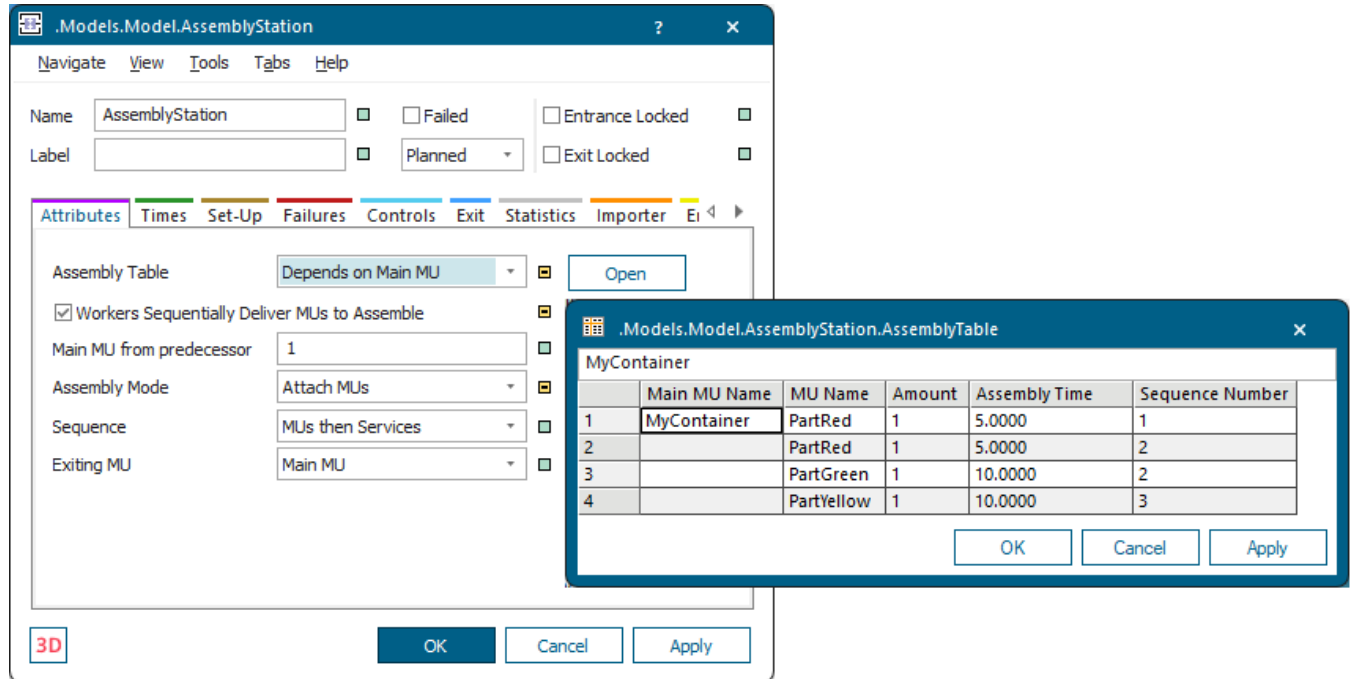


- The Worker picks the parts up at the Workplace attached to the Store, steps onto the Workplace attached to the AssemblyStation, and places the parts on the pallet.

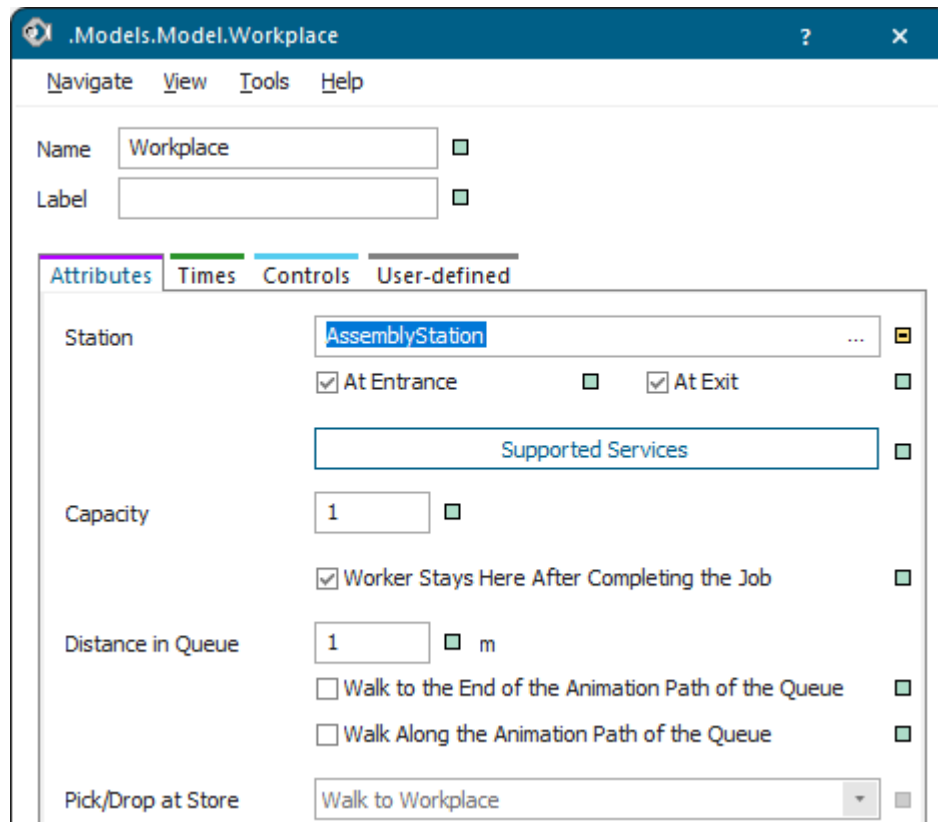
- We selected the settings below for the *AssemblyStation*.

We selected **Assembly Table** > **Depends on Main MU** and clicked **Open**. We typed in the actual assembly sequence into the column **Sequence Number**.

In our case the *Worker* first deposits a red part, followed by another red part, followed by a green part, and finally followed by a yellow part.



We use the *Workplace* attached to the *AssemblyStation* with its default settings.



- We use both the *Conveyor* transporting the parts to the *Drain* and the *Drain* itself with their default settings.

Go to [Configure the Store Which Provides the Parts](#)

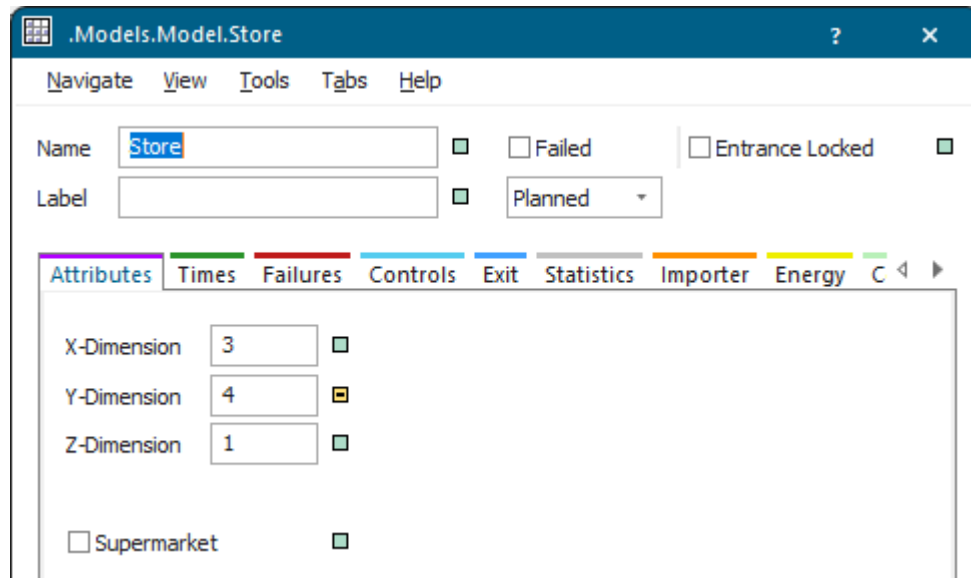
Go to [Watch how the AssemblyStation Assembles the Parts Delivered by the Worker](#)

Back to [Assemble Parts Sequentially with the Worker](#)

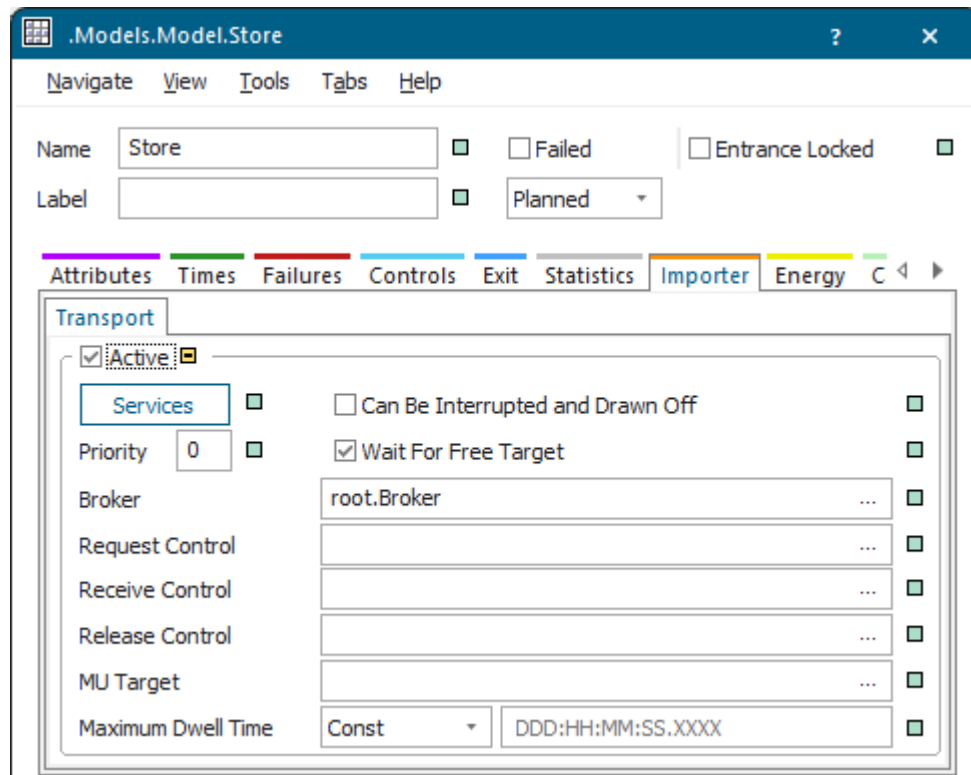
Configure the Store Which Provides the Parts

We configured the *Store*, which provides the parts to be placed onto pallets like this:

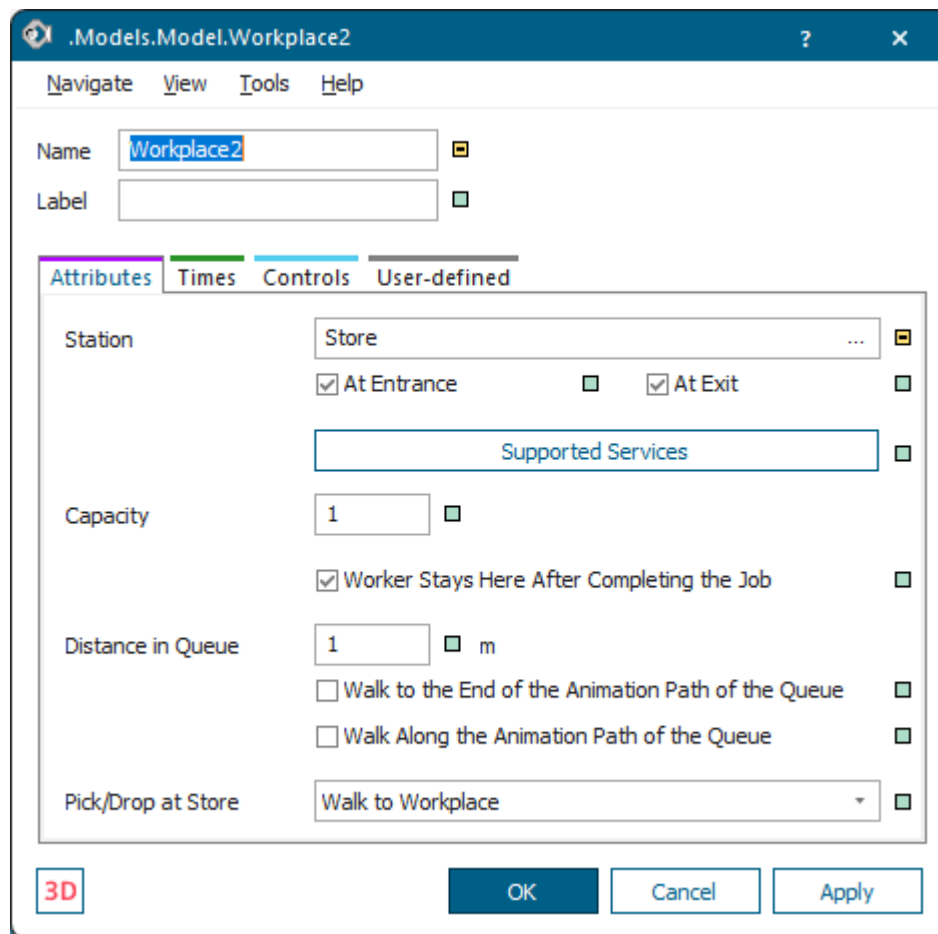
- We typed in 3 as the **X Dimension**, 4 as the **Y Dimension**, and 1 as the **Z Dimension**.



- We activated the **Transport Importer** and we want the *Worker* to **wait for a free target**.



- We configured the *Workplace* attached to the *Store*.



Back to [Configure the AssemblyStation to Which the Worker Delivers Parts](#)

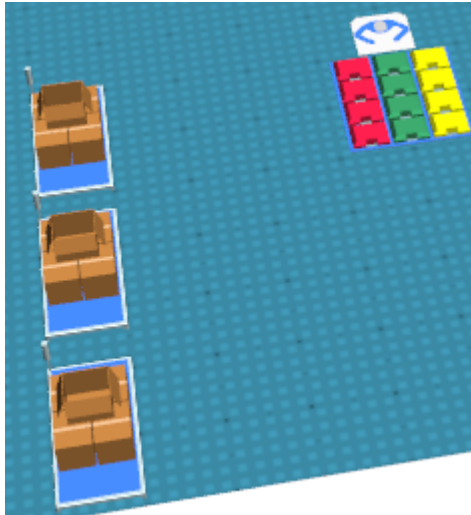
Go to [Watch how the AssemblyStation Assembles the Parts Delivered by the Worker](#)

Back to [Assemble Parts Sequentially with the Worker](#)

Watch how the AssemblyStation Assembles the Parts Delivered by the Worker

Run the simulation and watch how the *AssemblyStation* sequentially assembles the parts, which the *Worker* delivers

- First, the *Sources* produce four red, green, and yellow parts



- Then, the *Worker* steps onto the *Workplace* attached to the *Store* picks up a red part, and turns around. He then walks to the *Workplace* attached to the *AssemblyStation* and places the red part onto the pallet. He then walks back to the *Store* and picks up the parts in the sequence, which we entered into the **Assembly Table** and places the parts onto the pallet.

	Main MU Name	MU Name	Amount	Assembly Time	Sequence Number
1	MyContainer	PartRed	1	5.0000	1
2		PartRed	1	5.0000	2
3		PartGreen	1	10.0000	2
4		PartYellow	1	10.0000	3

OK Cancel Apply



To follow the assembly process step-by-step, open the *Event Debugger*.

Back to [Configure the AssemblyStation to Which the Worker Delivers Parts](#)

Back to [Configure the AssemblyStation to Which the Worker Delivers Parts](#)

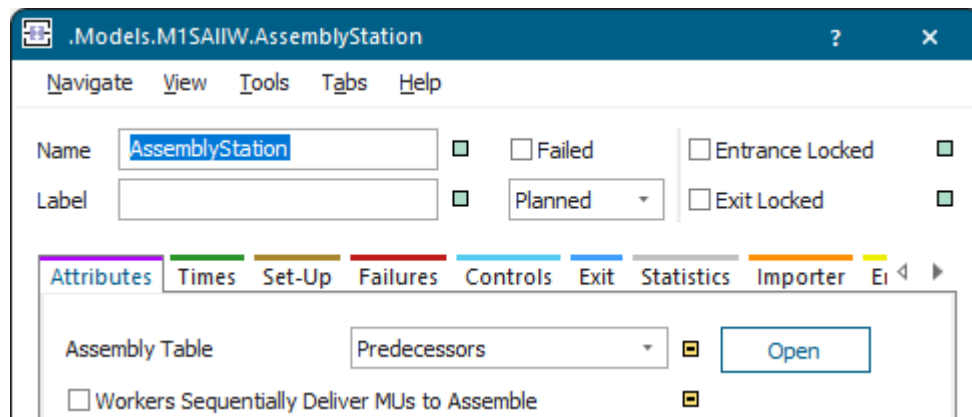
Back to [Assemble Parts Sequentially with the Worker](#)

Set How Workers Deliver Parts

Set How Workers Deliver Mounting Parts

You can set in the *AssemblyStation* how *Workers* will deliver the mounting parts that are to be assembled.

- To make several *Workers* deliver a part in parallel each, leave the check box **Workers Sequentially Deliver MUs to Assemble** cleared.



- To make a single *Worker* deliver the parts sequentially one after the other, select **Workers Sequentially Deliver MUs to Assemble**.

In our examples we will:

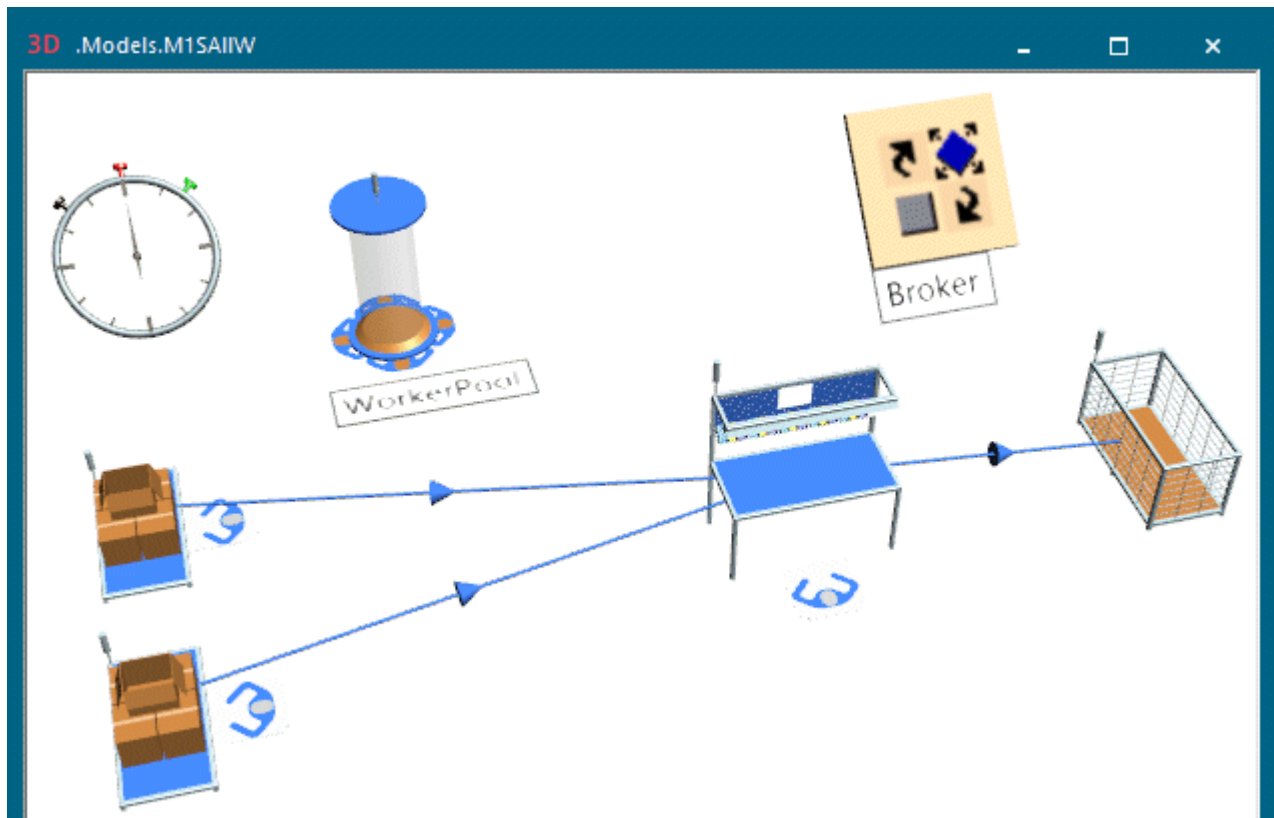
- [Make Several Workers Deliver Parts in Parallel](#)
- [Make a Single Worker Deliver Several Parts at Once](#)

Back to [Assemble Parts with the AssemblyStation](#)

Make Several Workers Deliver Parts in Parallel

In this simple model we make several *Workers* deliver the *mounting parts* to the *AssemblyStation* in parallel.

Our finished model looks like this.



Several Workers Deliver a Single Part Each

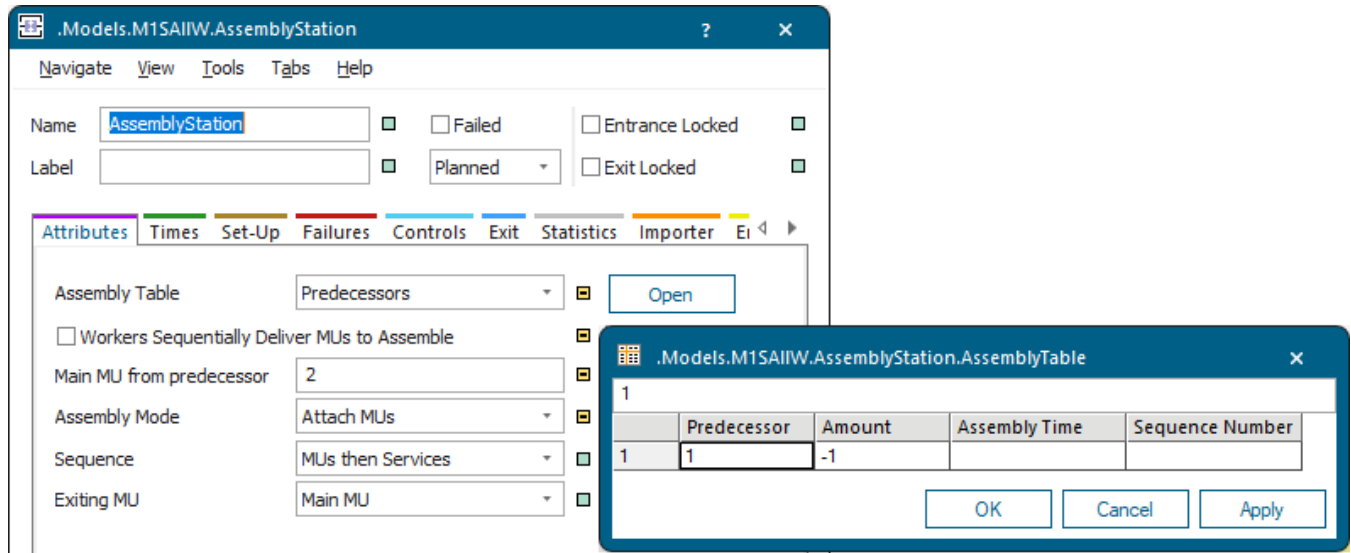
We configured the objects as follows to make several *Workers* deliver a single part each:

- The *SourcePallets* produces the **pallets**, the **main parts**, that our *WorkerMainParts* carries to the *AssemblyStation*.

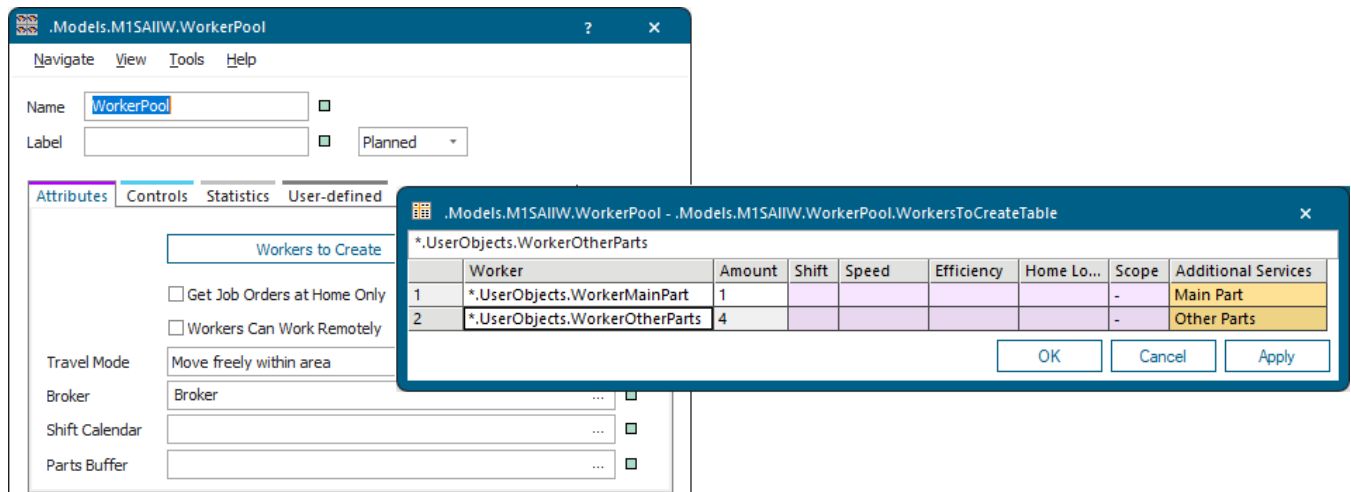
The *SourceParts* produces the **mounting parts** that our *WorkerOtherParts* carries to the *AssemblyStation* which loads them onto the *pallet*.

We use both with their default settings.

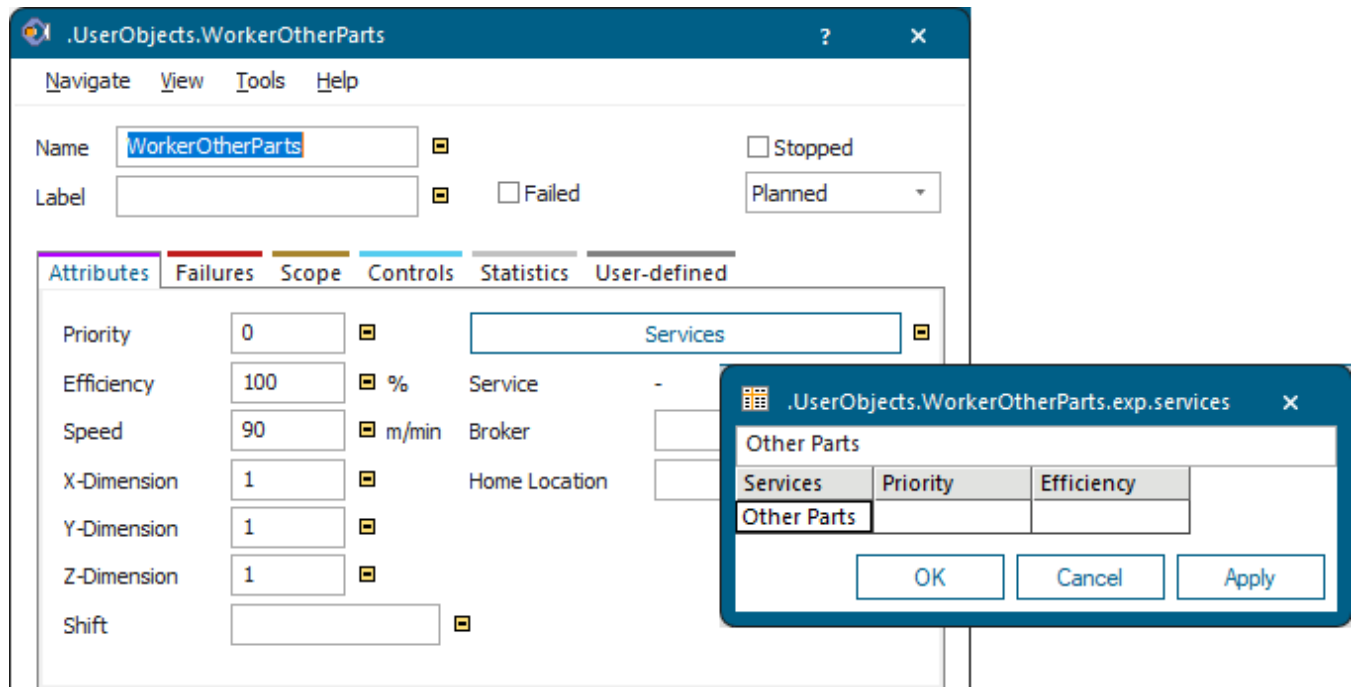
- For the *AssemblyStation* we selected these settings:



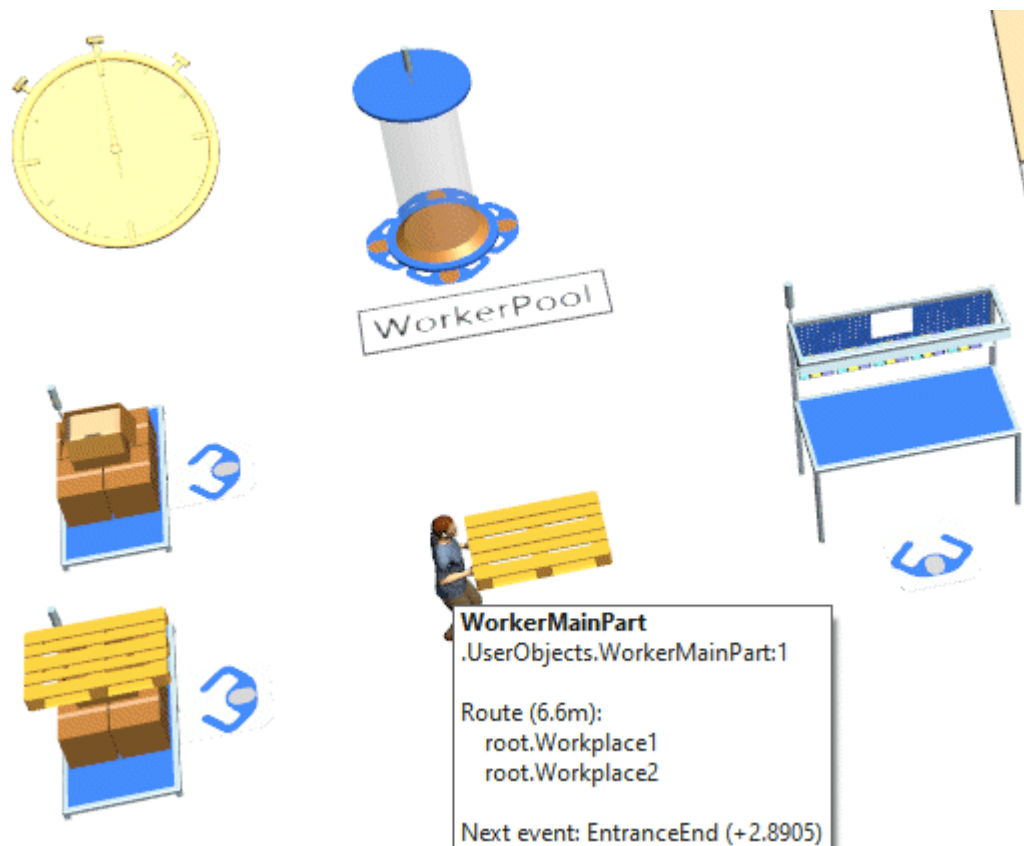
- We attached a *Workplace* each to the *SourceParts*, to the *SourcePallets*, and to the *AssemblyStation* where the four *Workers* of class *WorkerOtherParts* pick up the parts and place them respectively.
- For the *WorkerPool*, which creates our *Workers*, we selected these settings:



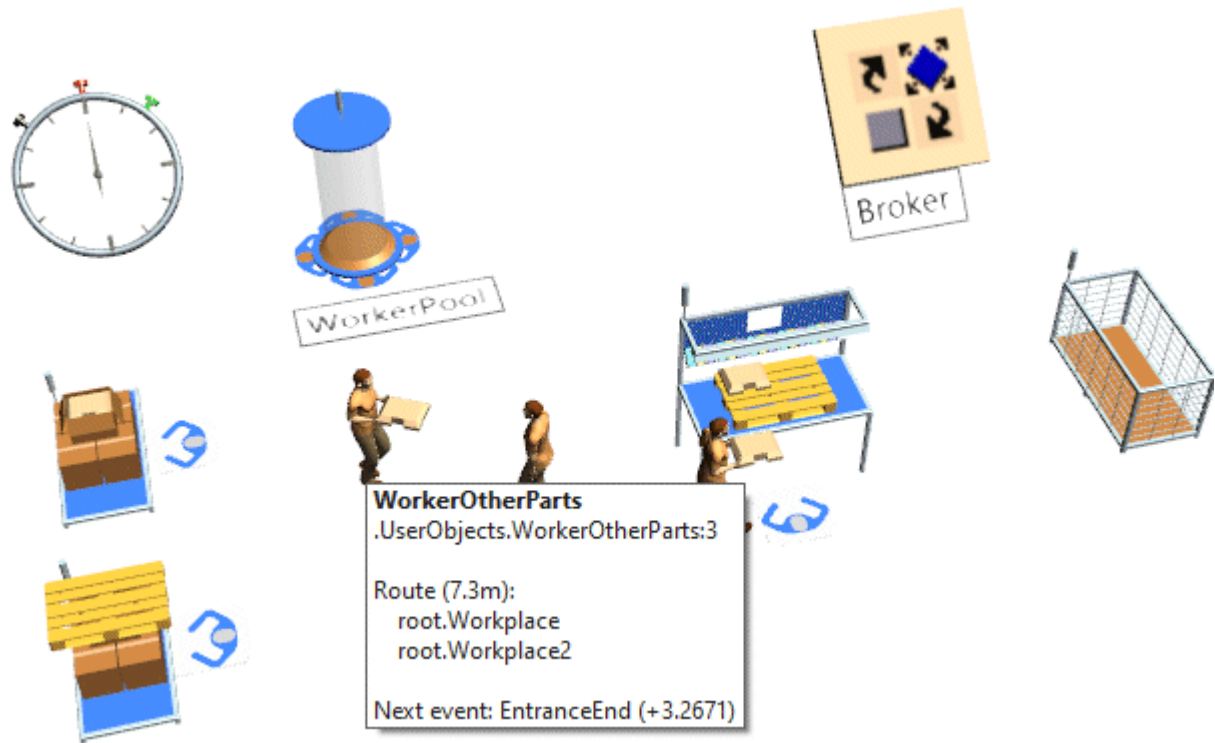
- For our *WorkerOtherParts* we selected these settings.



When we run the simulation, we see that our *WorkerMainPart* carries the *pallet*, the **main part**, to the *AssemblyStation* and places it there.

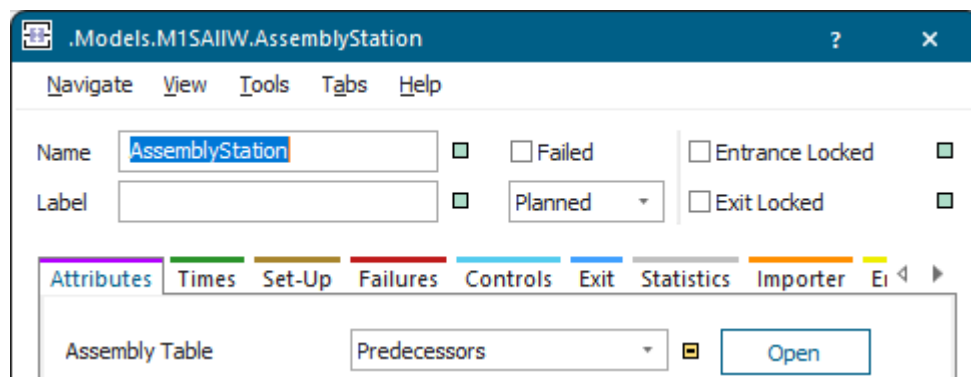


Meanwhile the *SourceParts* produces the parts. Four *Workers* named *WorkerOtherParts* walk one after the other from the *WorkerPool* to the *SourceParts*, step onto the *Workplace*, pick a part up, carry it to the *Workplace* attached to the *AssemblyStation* and place it onto the *pallet* located on the *AssemblyStation*.

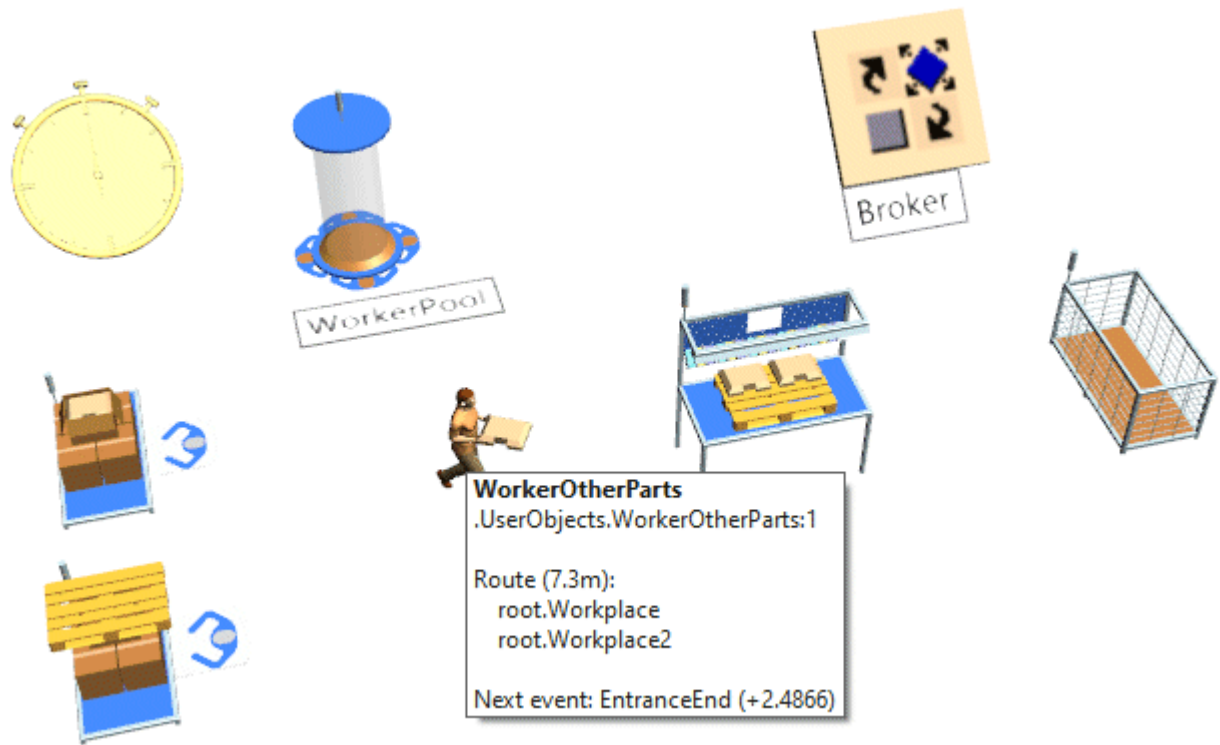


A Single Worker Delivers Part After Part

Let's now select **Workers sequentially deliver MUs to assemble** and watch what happens.



The *Worker* named *WorkerOtherParts:1* delivers four parts, one after the other, to the *AssemblyStation*, and returns to the *WorkerPool* once he's done.

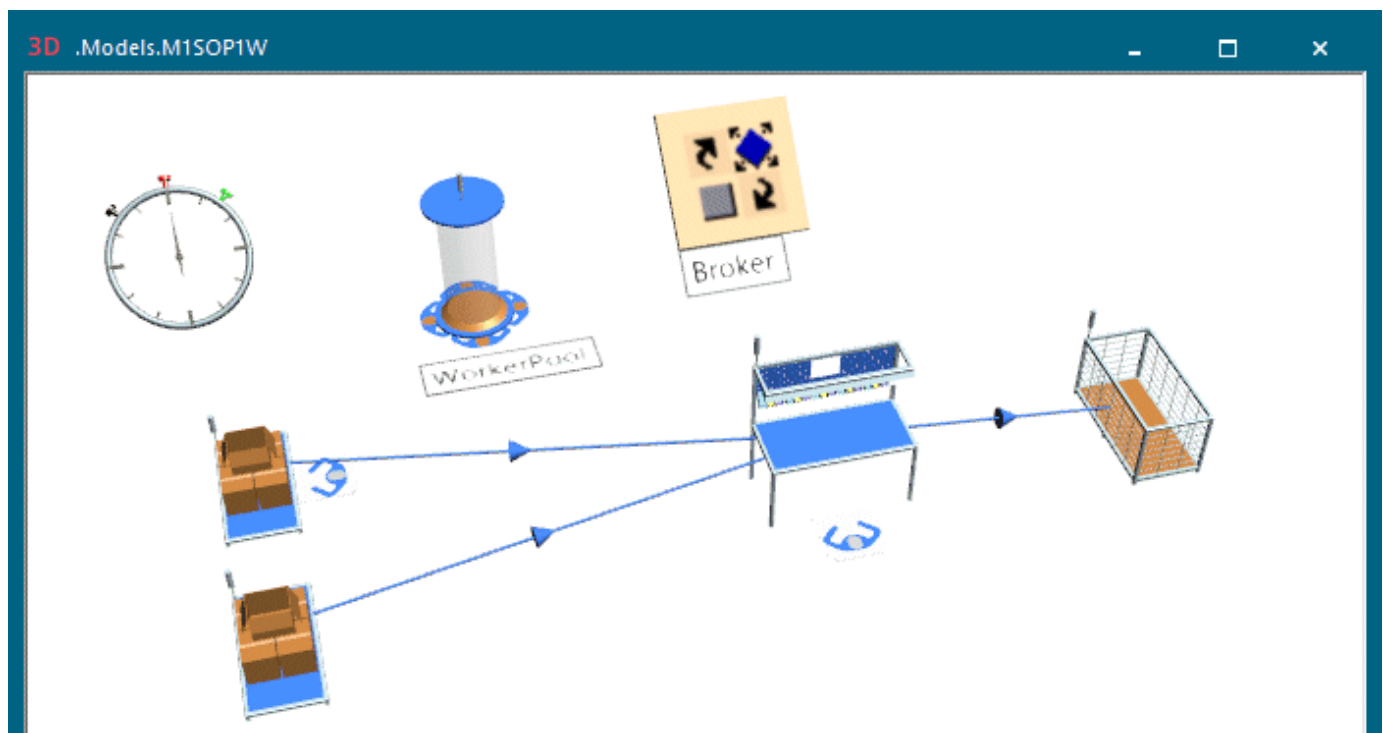


Back to [Set How Workers Deliver Mounting Parts](#)

Make a Single Worker Deliver Several Parts at Once

In this simple model we make a single *Worker* deliver as many *mounting parts* as he can carry to the *AssemblyStation*.

Our finished model looks like this.



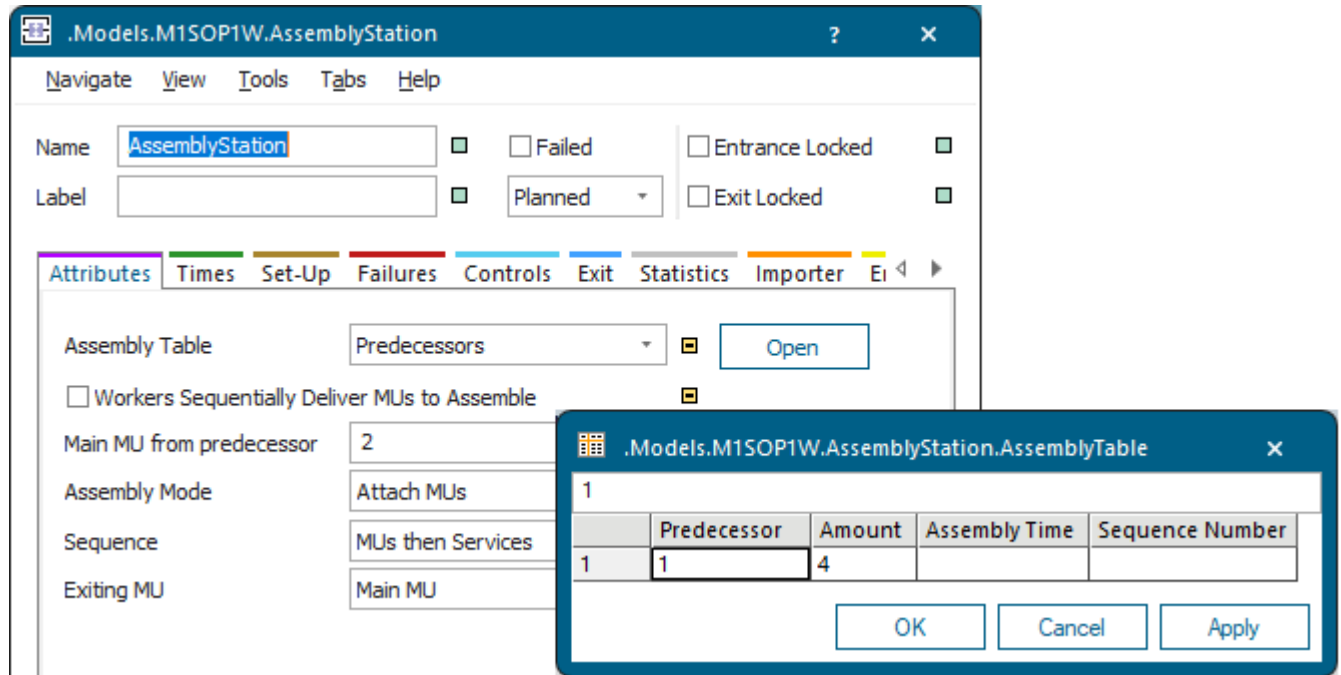
We configured the objects as follows:

- The *SourcePallets* produces the **pallets**, the **main parts**, that are automatically moved onto the *AssemblyStation*.

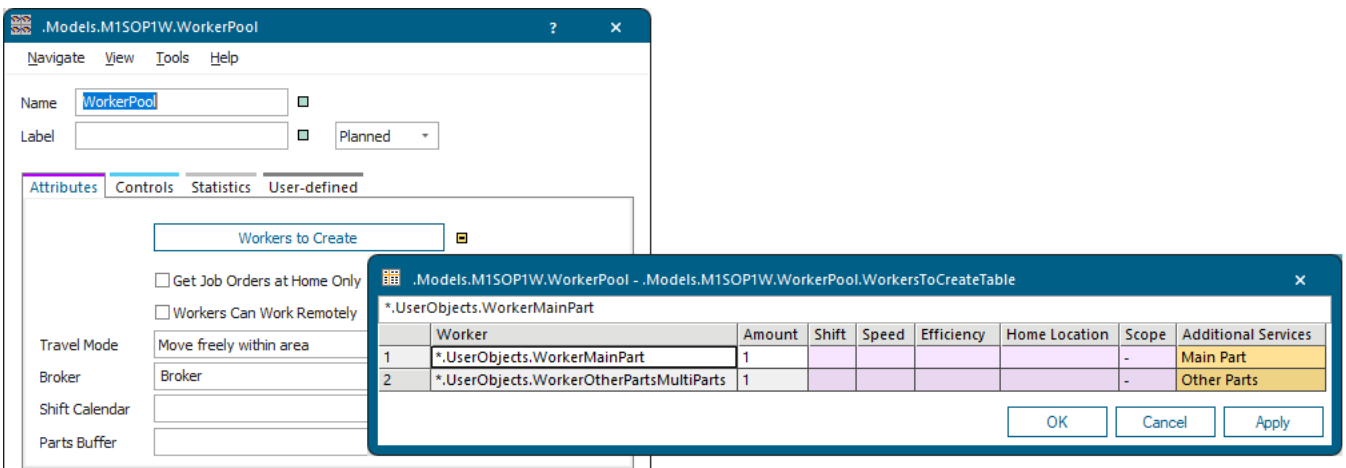
The *SourceParts* produces the **mounting parts** that our *Worker* carries to the *AssemblyStation*.

We use both with their default settings.

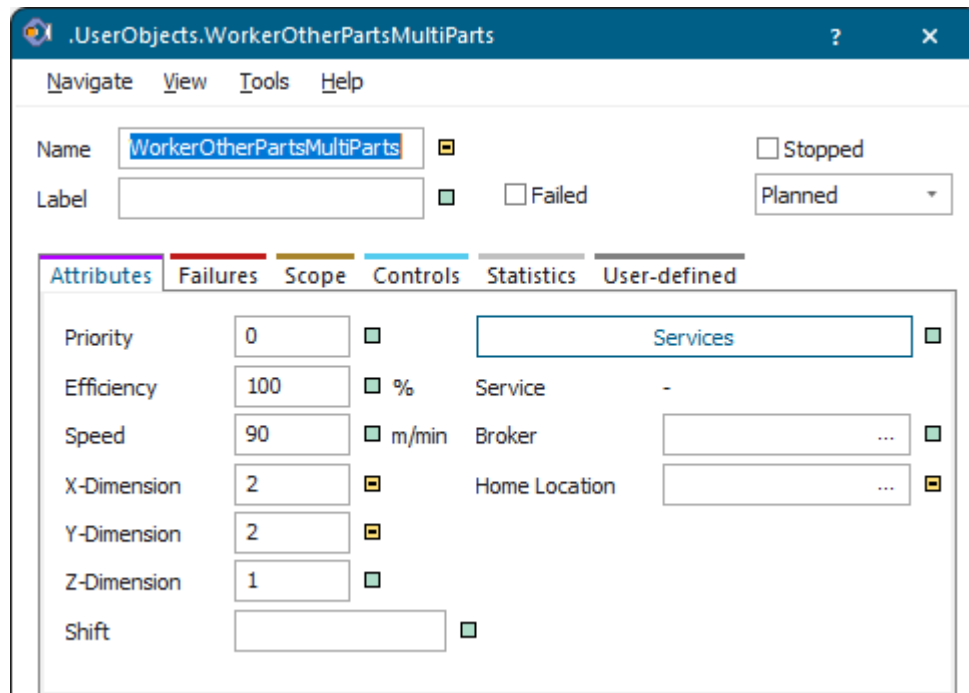
- For the *AssemblyStation* we selected these settings:



- We also attached a *Workplace* each to the *SourceParts* and to the *AssemblyStation* where the *Worker* picks up the parts and places them respectively.
- For the *WorkerPool*, which creates our *Workers*, we selected these settings:

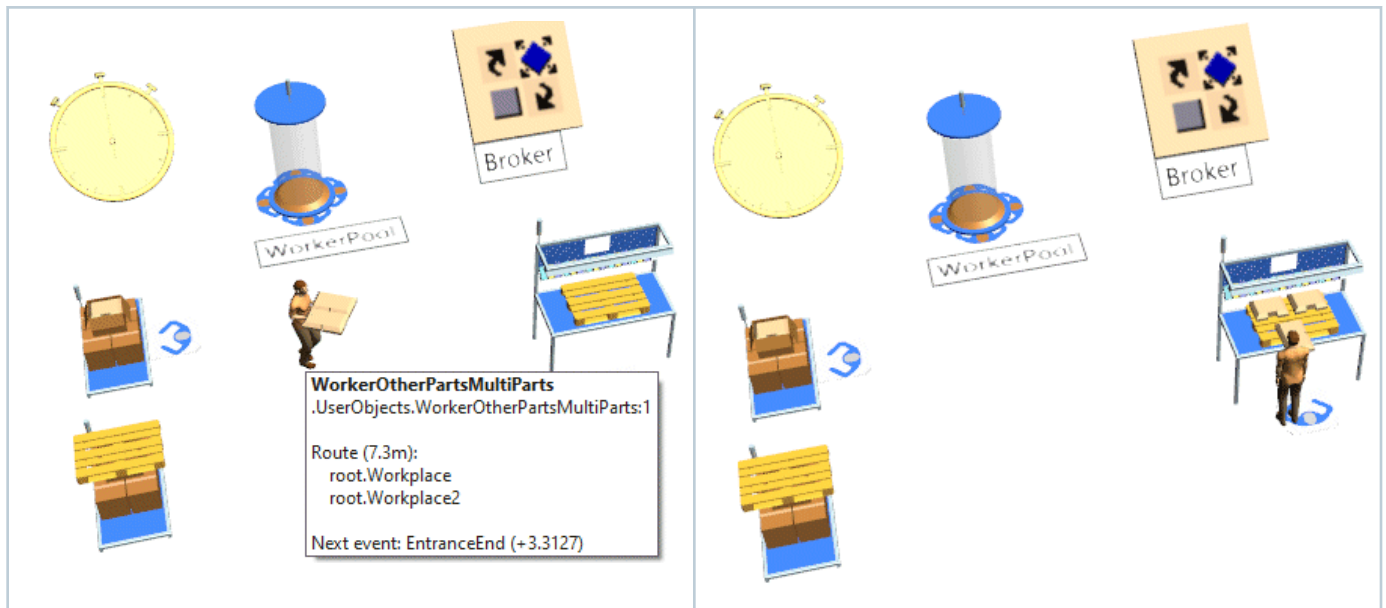


- For our *Worker* we increased the **X-Dimension** and the **Y-Dimension** to 2 each to allow him to carry four parts.



When we run the simulation, we see that the produced *pallet* is moved along the *Connector* to the *AssemblyStation*.

Meanwhile the *SourceParts* produces the parts. Our *Worker* walks to the *SourceParts*, steps onto the *Workplace*, waits until four parts are produced, picks them up one by one, carries them to the *Workplace* attached to the *AssemblyStation*, and places them onto the *AssemblyStation* one by one.



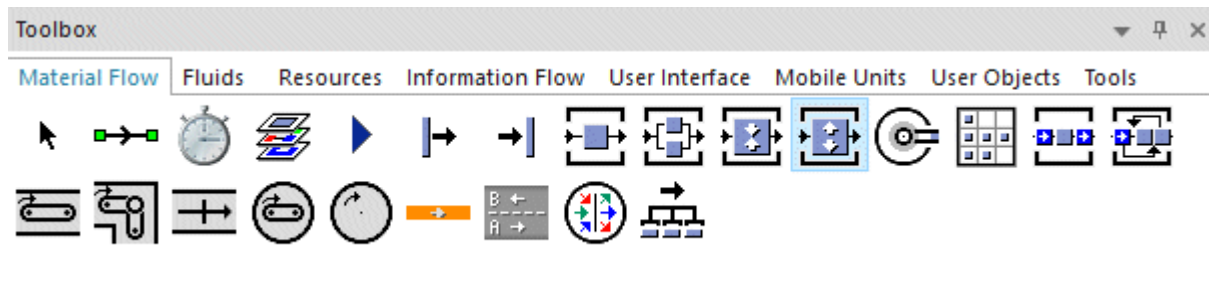
Back to [Set How Workers Deliver Mounting Parts](#)

Removing Parts with the Dismantle Station

Remove Parts with the Dismantle Station

The *DismantleStation* removes mounting parts from a main part.

You can insert the **DismantleStation**  into your simulation model from the folder **Material Flow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.

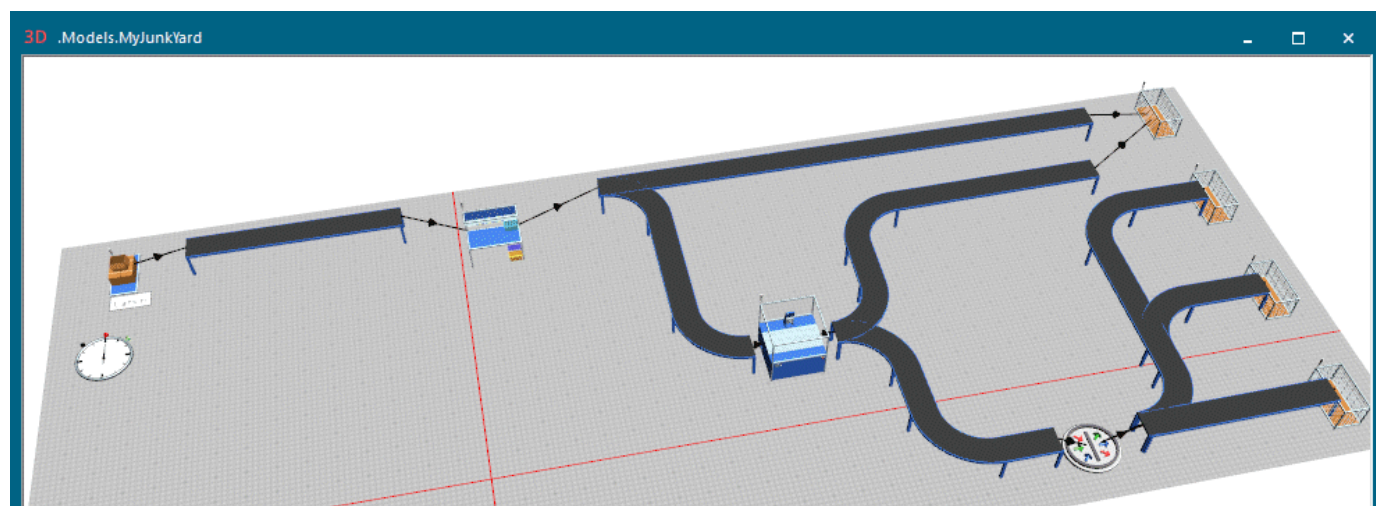


In this sample model we demonstrate how to remove parts from a car in a junk yard with the *DismantleStation*.

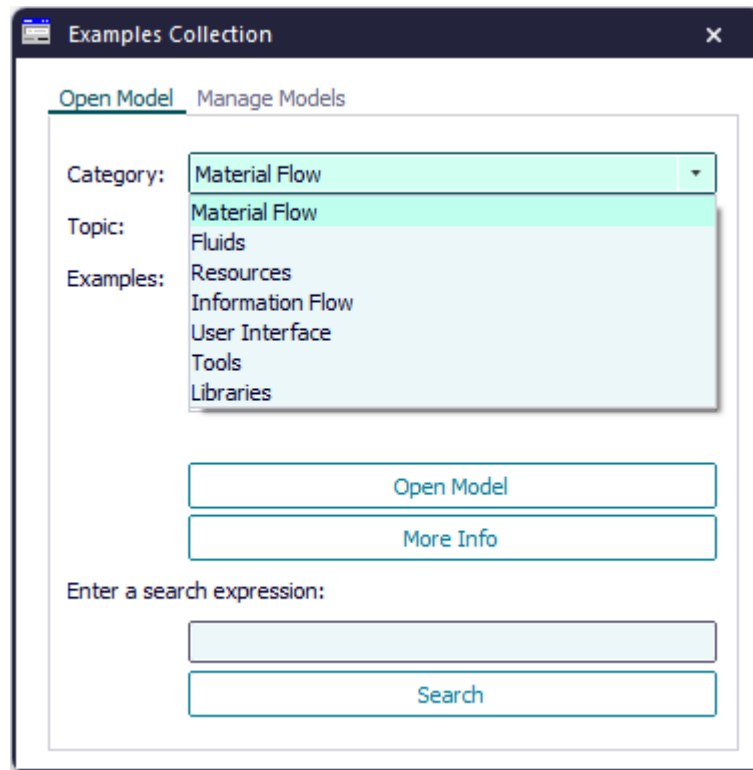
In our example we will:

- **Configure the Source Where the Cars Enter the Junk Yard**
- **Configure the Dismantle Station That Removes the Parts**
- **Configure the Station That Tests the Parts**
- **Configure the Station That Distributes the Removed Parts**

The finished simulation model looks like this:



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Back to [Modeling the Material Flow](#)

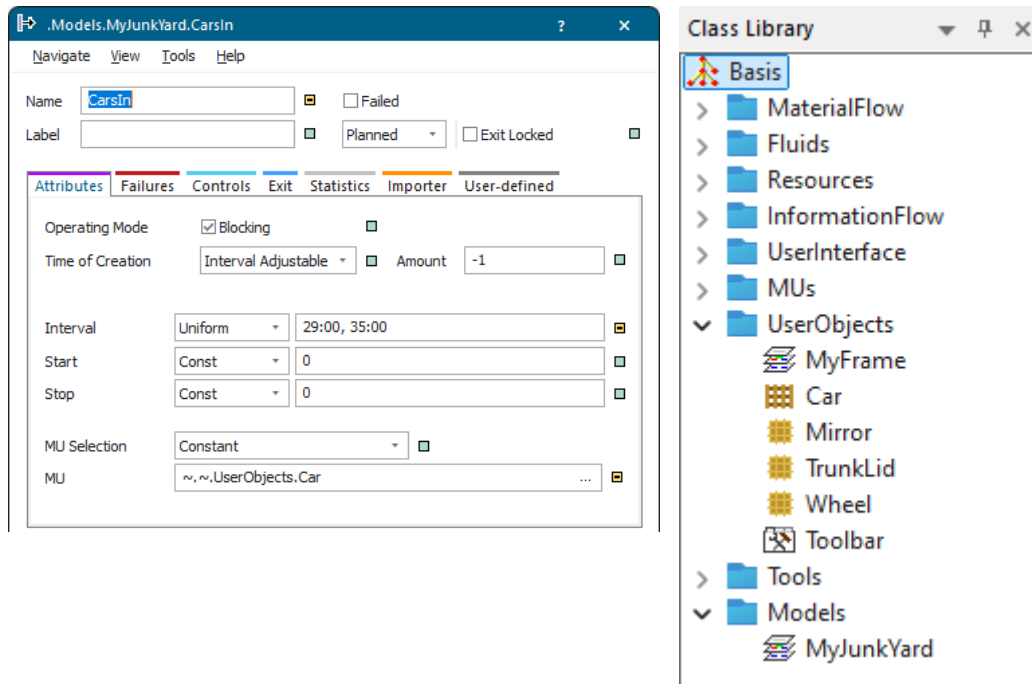
Configure the Source Where the Cars Enter the Junk Yard

The *Source* produces parts of type *Car* and moves them on a *Conveyor* to the *DismantleStation*, which takes them apart.

The *DismantleStation* removes the *trunk lid*, the outside *mirrors*, and the *wheels* and sends the stripped *car* on to the *scrap press*. The removed parts are sent on to the station *TestParts*, which checks if they can be resold or not. It sends the good parts on to the respective *Drain*, which removes them from the scrap yard proper. The bad parts, on the other hand, are moved to the *scrap press*.

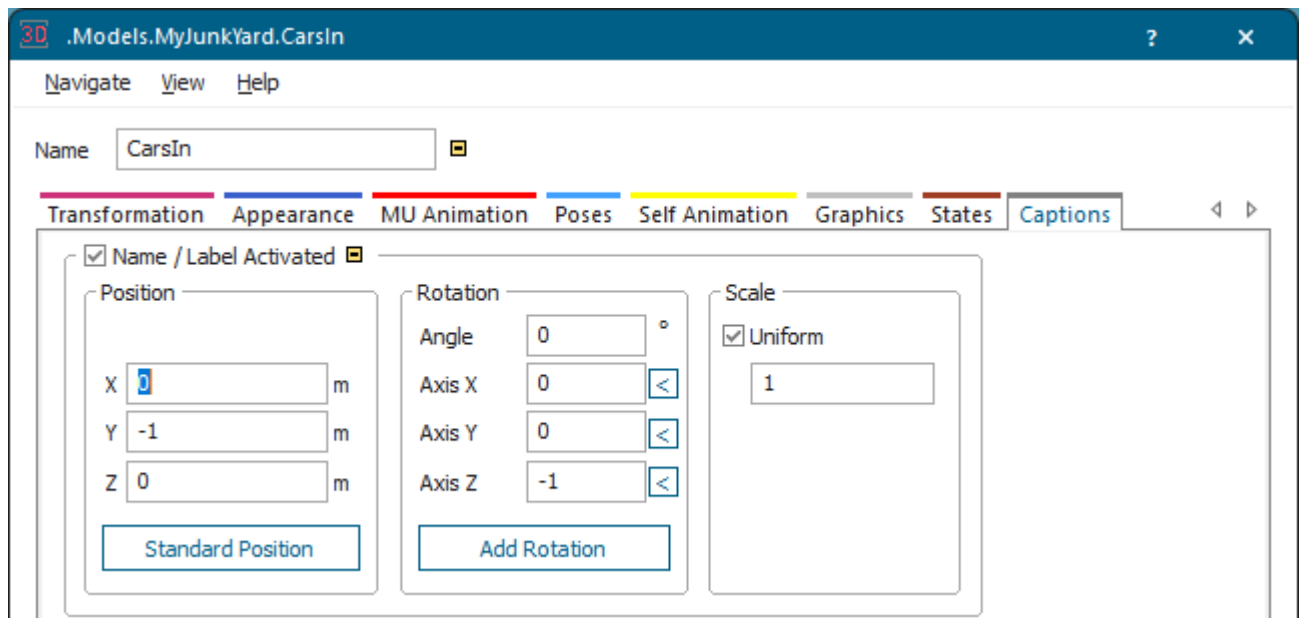
Insert and configure the *Source* which produces the parts:

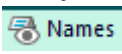
- Select how the *Source CarsIn* produces the parts. We selected an **adjustable interval** and the **uniform distribution**.
- Select the type of part which the *Source* is to produce. We selected *Car* in our example.



With these settings the *Source CarsIn* creates cars whose inter-arrival time is **uniformly** distributed between 29 and 35 minutes.

We activate **Captions** of the objects to allow us to identify them. To do so, we click the object and press the **spacebar**. We change to the tab **Captions** and activate **Name/label activated**.



We do this for all objects, which we insert into the model. We can hide or show all captions by clicking the button **Names**  on the **View** ribbon tab

Back to Remove Parts with the Dismantle Station

Configure the Dismantle Station That Removes the Parts

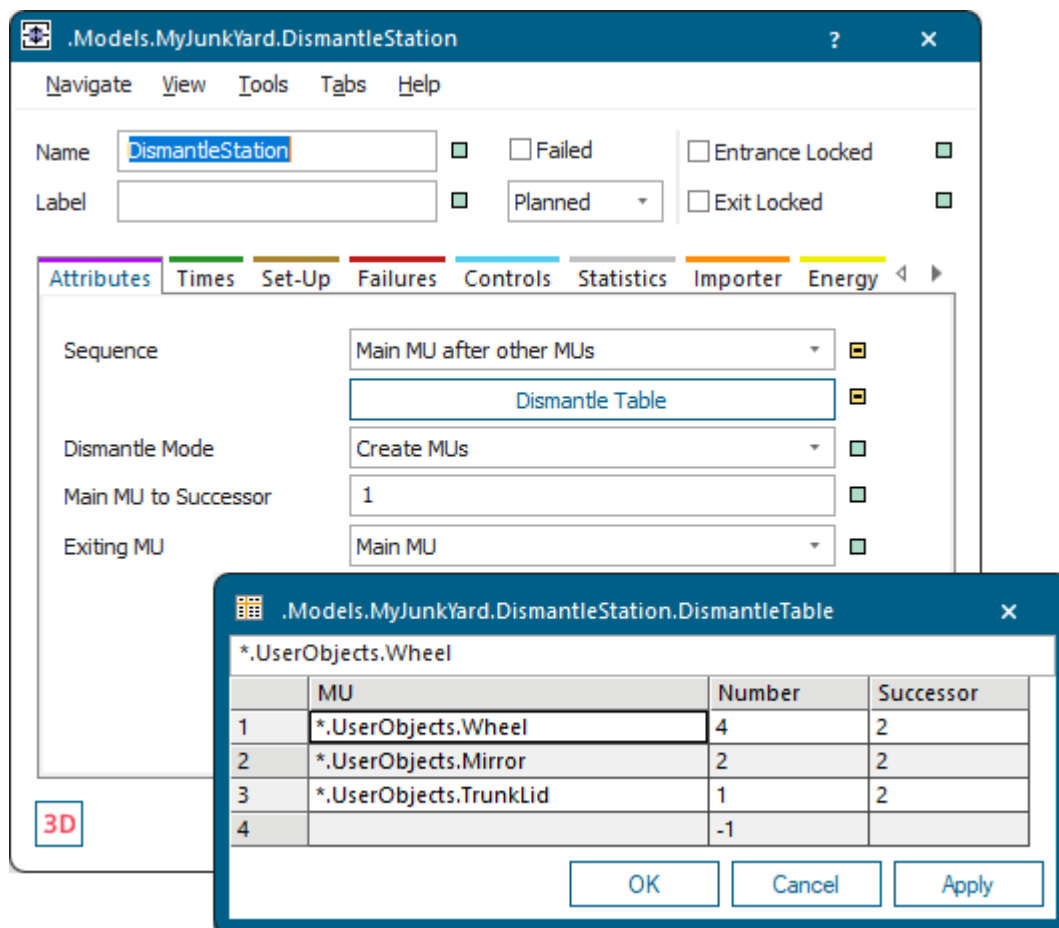
Insert and configure the *DismantleStation* which removes the mounting parts from the main part.

In our example the *DismantleStation* removes the *trunk lid*, the outside *mirrors*, and the *wheels* from the car:

- Select the **Sequence** in which it moves the parts on to its successors. We selected **Main MU after other MUs**, which means that it first moves the mounting parts—trunk lid, outside mirrors, wheels in our case—on to the successor, and then the main *MU*, the stripped car in our case.

We then clicked **Dismantle Table** and entered the required information about the mounting parts which we want to remove:

- We want to remove 4 wheels and send them on to the successor with the number 2 of the *DismantleStation*.
- We want to remove 2 mirrors and send them on to the successor with the number 2.
- We want to remove 1 trunkLid and send it on to the successor with the number 2.



We assign the dismantled parts **wheel**, **mirror**, and **trunk lid** a different material each as we do not have a graphic for them at the moment.

We click the object with the right mouse button on the *Class Library* and select **Open in 3D**. We click the graphic named **Default** of the object and press the **spacebar**. We change to the tab **Material**, select **Material active**, and select a color. Our **wheel** is blue, our **mirror** is red, and our **trunk lid** is green. This then looks like this:

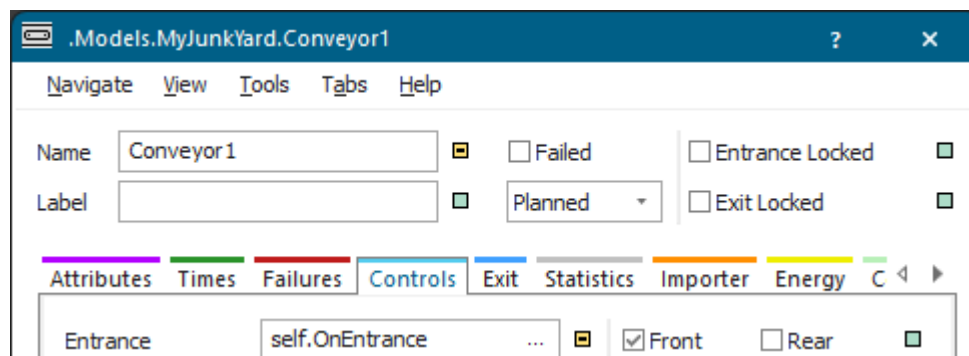


- Select the **Dismantle Mode**. We used the default setting **Create MUs**; the *DismantleStation* then creates a new *part* for each successor and moves that *part* there.
- Select the **successor** to which the **main MU**, the stripped car in our case, moves. We used the default setting **1**.
- Select if the *DismantleStation* moves the main *MU* or a new *MU* in to its successor. We used the default setting **Main MU**.

As removing the parts also takes up some time, we entered 30 minutes as the **Processing Time** on the tab **Times**.

As we want to also show that the parts were removed from the car, we drew an icon of the car without these parts. This icon has the number 1.

To make the successor line display the picture of the stripped car, we created an **Entrance Control** for this *Conveyor* and programmed it to switch to the icon with the number 1 once the car moves onto it.



```
@.CurrIconNo := 1
```

The result looks like this:

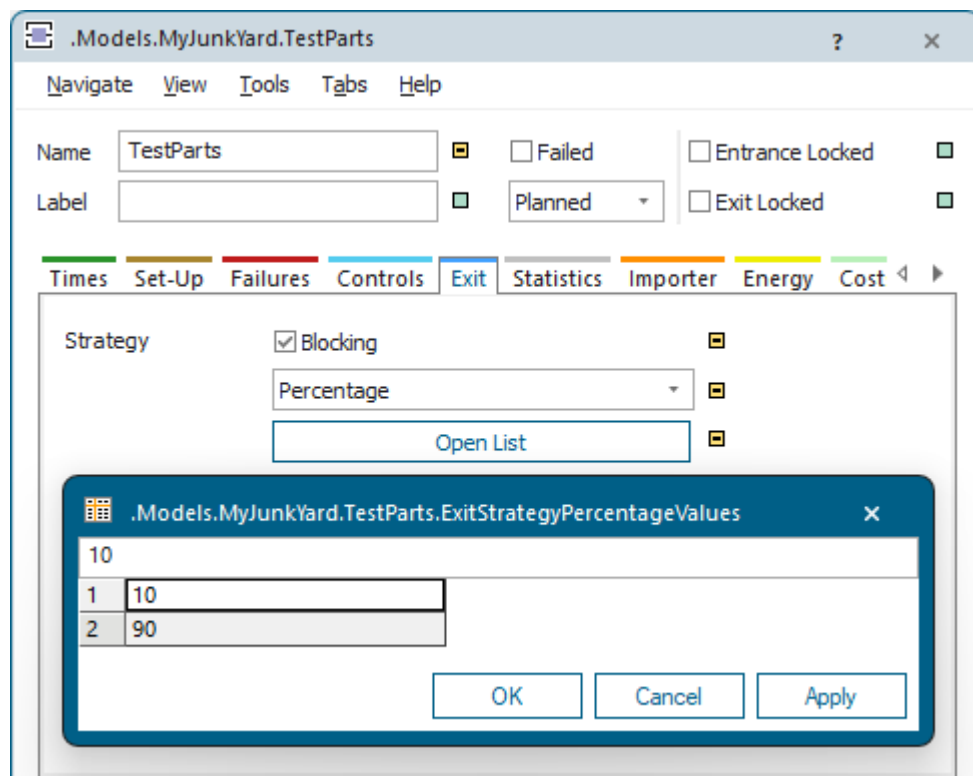
[Back to Remove Parts with the Dismantle Station](#)

Configure the Station That Tests the Parts

Insert and configure the station which checks if the quality of the parts is good enough to be sold or if they have to be scrapped.

In our example we used a *Station* which we named *TestParts*. Except for the **Exit Strategy** you can use it with its default settings.

- As we want a certain percentage of the parts to be bad, we selected the strategy **Percentage**. The test station then moves the parts on to its successors according to a percentage distribution.
- We then clicked **Open List** and entered the percentages of the parts that are sent on to the two successors.
 - 10 percent of the parts are sent on to the successor with the number **1**. These parts are bad and have to be scrapped; they end up in the *scrap press*.
 - 90 percent of the parts are sent on to the successor with the number **2**. These parts are good enough to be sold.



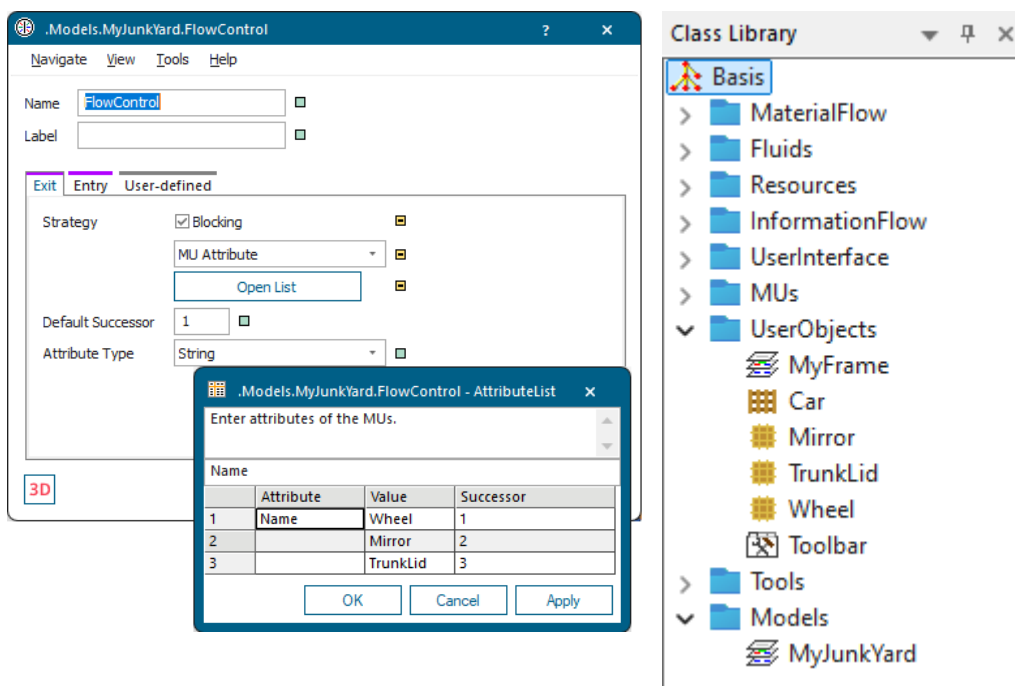
[Back to Remove Parts with the Dismantle Station](#)

Configure the Station That Distributes the Removed Parts

Insert and configure the station which distributes the removed parts according to their types.

We decided to use a *FlowControl* which moves the parts on to the type-specific *Drains* according to an attribute of the respective part.

- We selected the **Exit Strategy > MU Attribute**.
- We then clicked **Open List** and entered the required information into the **Attribute List**. We used the attribute **Name** of all parts. For this reason we only have to enter **Name** once into the column **Attribute** and can leave these cells empty for the remaining two parts.
 - We entered **Name**, followed by the **value** of the attribute, namely **Wheel**, followed by the number of the **successor** onto which the part is to be moved, namely **1**.
 - We then entered **Mirror**, followed by the number of the **successor**, namely **2**.
 - We finally entered **TrunkLid**, followed by the number of the **successor**, namely **3**.



Start the simulation and watch how the *DismantleStation* removes the *trunk lid*, the outside *mirrors*, and the *wheels* from our *car*. The stripped *car* is sent straight on to the *scrap press*, while the removed parts are sent to the *test station* which checks if they are good or bad. The bad ones are sent to the *scrap press*, while the parts, which can be resold, are distributed to the respective *Drains*.

You can also view the *Statistics Report* at the end of the simulation run (**F6**) which contains detailed statistics information.